

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

CROSS-PART GOVERNING AMENDMENT

Version 1.4

RUN THE MAINLINE TRAIN

Semantic Evidence Preservation, Impact-Selected Acceptance, Prepared Immutable Runtime Layers, Tree-Bound Finalization, and Speculative Mainline Integration

Governing Principle

Prove everything that matters. Preserve proof only when its meaning is demonstrably unchanged. Test exactly what semantic change can affect. Prepare immutable infrastructure once. Qualify the predicted integration state before the train reaches it. Bind the exact merged result without replaying the entire voyage.

Version 1.4 • August 13, 2026 • Prompt 0 Integrated Edition

Status: Governing amendment candidate; Prompt 0 current-state survey and requirements register incorporated; authoritative only after protected-mainline acceptance

Document ID: CS-SL-XP-001 v1.4-R1

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Cross-Part Governing Amendment - Version 1.4, "Run the Mainline Train" - Prompt 0 Integrated Edition
Base authority	Part I v1.0 plus accepted amendments through v1.2; Part II v1.0 plus accepted amendments; Part III v1.0 plus accepted amendments through v1.3; current effective authority index controls at implementation time.
Version	1.4-R1 (governing version remains 1.4)
Date	August 13, 2026
Document ID	CS-SL-XP-001 v1.4
Status	Governing amendment candidate; Prompt 0 verified baseline incorporated; becomes authoritative only through protected-mainline acceptance.
Amendment trigger	Prompt 0 verified that ordinary mainline currently selects 38 required suites and no conditional suites, consumes 1,180 s wall time and 84.0 summed worker-minutes in representative authority run 31728114656, and repeats bootstrap work that materially exceeds suite execution time.
Primary objective	Replace fresh-only broad ordinary mainline proof with fail-closed semantic evidence closure, impact-selected obligations, verified immutable prepared layers, tree-bound finalization, and speculative ordered train qualification without weakening release-candidate proof.
Implementation shape	One v1.4 implementation branch, one implementation pull request, one legacy/current broad cutover acceptance, one protected merge. Internal stages are not independently merged.
Future compatibility	Self-hosted/prewarmed worker pool is explicitly supported as a later acceleration layer but is not required for v1.4 acceptance.

Prompt 0 surveyed baseline origin/main 0055d012a121a8950b7fa70d371d5eafc6223d10, surveyed August 13, 2026; worktree 0 ahead / 0 behind.

Prompt 0 effective authority Part I v1.2 / Part II v1.2 / Part III v1.3; protected check: Sounding Line / Mainline Decision; only scripts/sounding-line/finalizer.mjs may emit RELEASE_GO through explicit frozen-candidate workflow_dispatch.

Prompt 0 source records Project_Sounding_Line_v1.4_Current_Authority_Survey.md and Project_Sounding_Line_v1.4_Governing_Requirements_Register.md. Incorporated as authoring evidence, not independent runtime authority.

Normative incorporation rule

This amendment is additive and superseding only where it explicitly changes current Sounding Line semantics or runtime obligations. It does not reduce coverage. It converts existing v1.2 proof-efficiency requirements into a concrete mainline-throughput architecture and preserves Part III v1.3 workspace-cleanup law. When this document conflicts with a later accepted authority index or amendment, the accepted authority index and later amendment control.

A.1 Three-document authority split remains intact

Authority domain	v1.4 responsibility
Part I - Verification meaning	Semantic dependency fingerprints, evidence preservation, minimum sufficient evidence, gate scope, proof equivalence, tree-bound decision semantics.
Part II - Execution machinery	Prepared artifact lifecycle, worker-layer restoration, train planning/execution mechanics, resource brokerage, isolation, cleanup, durable scheduling, distributed execution.
Part III - Enforcement	Mainline-train admission, merge order, protected binding, shadow/cutover policy, Codex behavior, incident handling, rollback, authority-index changes, completion wording.

B. Contents

1. Executive Summary	
2. Amendment Trigger and Current Throughput Problem	
3. Goals, Non-Goals, and Success Criteria	
4. Supersession Map and Cross-Part Boundaries	
5. Canonical v1.4 Vocabulary	
6. Non-Negotiable Invariants	
7. Target Architecture Overview	
8. Semantic Evidence Fingerprint	
9. Contract Closure and Transitive Dependency Resolution	
10. Evidence Preservation, Rebinding, and Invalidation	
11. Legacy v1.3 Evidence Adoption	
12. Impact-Selected Ordinary Mainline Gate	
13. Always-Fresh Sentinel Spine	
14. Release-Candidate Separation	
15. Prepared Immutable Runtime Layers	
16. Prepared Artifact Identity and Security	
17. Transport, Storage, and Lifecycle of Prepared Layers	
18. Worker Bundling and Per-Suite Receipt Boundaries	
19. Git Source Identity and Tree Identity	
20. Ultra-Light Protected Binding	
21. Mainline Train Model	
22. Train State Machine and Admission	
23. Candidate Mutation, Withdrawal, and Replanning	
24. Merge Conflicts, Migrations, and Shared Schema Changes	
25. Fairness, Priority, and Starvation Prevention	
26. Security Threat Model	
27. Privacy, Secrets, and Supply-Chain Boundaries	
28. Throughput Metrics and Performance SLOs	
29. Part III v1.3 Workspace-Lifecycle Compatibility	
30. Shadow Mode and Decision Equivalence	
31. Atomic Authority Cutover	
32. Rollback and Emergency Serial Mode	
33. Internal v1.4 Implementation Stages	
34. Acceptance Matrix and Exit Criteria	
35. Machine-Readable Policy and Registry Changes	
36. Failure and Conformance Codes	
37. Paused-Fleet Evidence Adoption	
38. Future Option C: Prewarmed Self-Hosted Workers	
39. Explicit Non-Goals	
40. References and Final Governing Rule	
Appendix A. Evidence Fingerprint Schema	
Appendix B. Train Receipt and State Schemas	
Appendix C. Prepared Artifact Identity Examples	
Appendix D. Detailed Acceptance Scenarios	
Appendix E. Threat and Abuse Matrix	
Appendix F. Normative v1.4 Closure Checklist	
Appendix G. Cross-Part Normative Requirements Register	
Appendix H. Mainline Train Algorithm and Decision Procedure	
Appendix I. Prepared Layer Operational Lifecycle	
Appendix J. GitHub and Branch Protection Integration Contract	

Appendix K. v1.4 Regression and Negative Test Catalog

Appendix L. Operational Handoff and Paused-Fleet Resume Matrix

Appendix M. v1.4 Implementation Evidence Ledger

Appendix N. Representative Cross-Project Train Examples

Appendix O. Human, Codex, Scheduler, and Release-Authority Responsibilities

Appendix P. Atomic Cutover Runbook

Appendix Q. Prompt 0 Verified Current-State Baseline

Appendix R. Prompt 0 Governing Requirements Traceability Register

1. Executive Summary

Project Sounding Line v1.4 is a throughput amendment to the repository's existing verification authority. Its purpose is not to make testing less strict. Its purpose is to stop paying repeatedly for proof whose protected meaning has not changed, stop provisioning expensive resources that a selected suite does not need, and stop forcing every qualified phase candidate to wait for earlier candidates to finish before it can prove the integration state it will actually encounter.

Versions 1.1 and 1.2 already established the essential law: SHA inequality is not semantic invalidation; minimum sufficient evidence is mandatory; preserved evidence must remain source-bound and auditable; undeclared setup and unexplained overvalidation are conformance defects; mainline is permitted to combine fresh, preserved, and rebound evidence; and canonical merge ownership serializes the final integration transition rather than unrelated development. Version 1.4 turns those principles into an explicit mainline-train architecture with content-addressed prepared runtime layers and tree-bound finalization.

The pre-v1.4 system may still be correct while being operationally wasteful. A normal candidate can cause dozens of hosted workers to repeat checkout, Node setup, dependency installation, Prisma generation, database work, browser provisioning, and artifact upload. Broad path-based invalidation can also cause unrelated project movement to erase usable proof. When eight projects produce phase candidates faster than one mainline lane can digest them, this becomes the primary integration bottleneck rather than an incidental inconvenience.

Prompt 0 converted that hypothesis into measured repository truth. Against origin/main 0055d012a121a8950b7fa70d371d5eafc6223d10, the current mainline planner selects exactly 38 required nodes in three parallel waves and no conditional nodes. Representative authority run 31728114656 completed in 1,180 seconds wall time while consuming 5,042 worker-seconds (84.0 worker-minutes). Summed npm ci, checkout, setup-node, and declared preparation consumed 3,535 seconds, compared with 1,161 seconds of actual suite execution. The amendment therefore treats repeated bootstrap and fresh-only evidence production as measured primary throughput costs, not speculative inconveniences.

v1.4 outcome

Ordinary mainline acceptance becomes an evidence-closure and integration operation. Sounding Line computes the mandatory obligations, carries forward proof whose semantic dependencies are unchanged, executes only fresh affected obligations and global sentinels, restores content-addressed prepared layers instead of rebuilding the same environment repeatedly, qualifies downstream candidates against predicted integration trees, and performs a final protected binding that is primarily identity, provenance, tree, evidence, and cleanup reconciliation.

Figure 1. Sounding Line v1.4 acceptance architecture

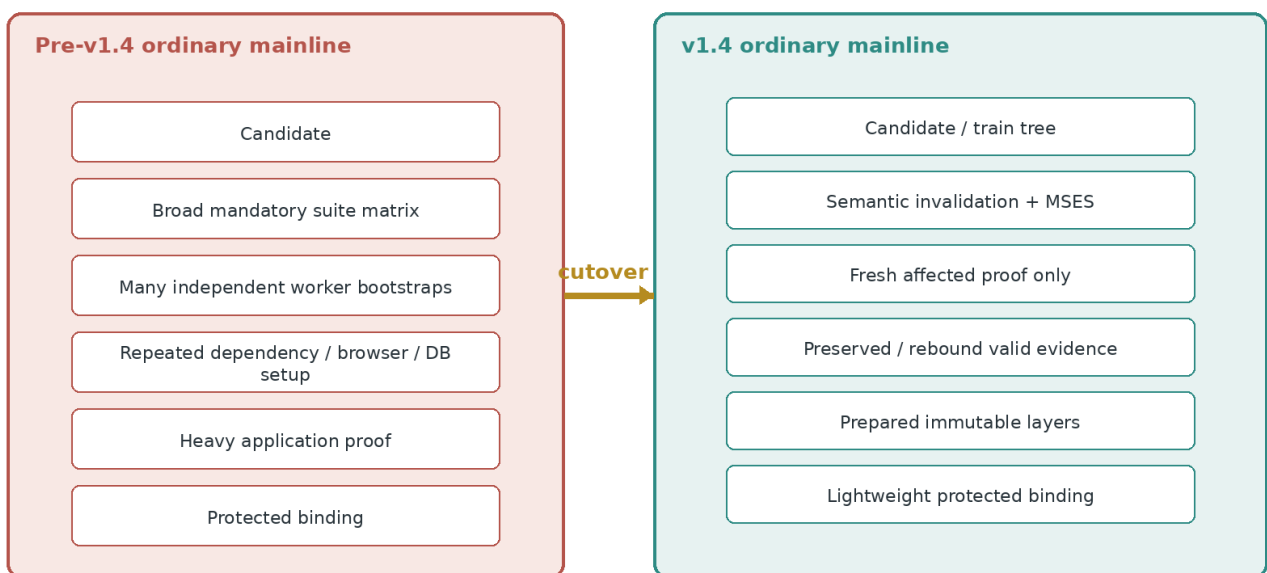


Figure 1. Pre-v1.4 broad repeated execution versus the v1.4 minimum-sufficient-evidence mainline path.

2. Amendment Trigger and Current Throughput Problem

The operational trigger is the current parallel-development model: multiple project phases may be implemented concurrently, but protected mainline remains one authoritative history. That architecture is sound. The problem is the amount of repeated work performed at the boundary. Phase implementation now commonly finishes faster than mainline finalization can consume qualified candidates, creating a tail queue measured in hours even when the code itself required less than an hour.

Prompt 0 verified the exact pre-amendment ordinary-mainline shape. `testing/release-gates.json` declares 38 required and 14 conditional mainline suites. `planner.mjs` currently selects all 38 required IDs and none of the conditional IDs, producing 18 parallel nodes in wave 0, 17 in wave 1, 3 in wave 2, and 0 exclusive nodes. The planner receives no changed-path input for ordinary mainline selection, so “conditional” is policy vocabulary rather than an impact-selected decision. No ordinary mainline receipt is currently preserved across authoritative candidates.

2.1 Prompt 0 Verified Authority Baseline

The following observations are the verified pre-cutover baseline for design and acceptance comparison. Later repository movement may update paths or counts, but implementation-time drift must be remeasured and recorded rather than silently replacing this baseline.

Concern	Prompt 0 verified current truth
Surveyed source	origin/main 0055d012a121a8950b7fa70d371d5eafc6223d10; August 13, 2026
Effective governing chain	Part I v1.2 / Part II v1.2 / Part III v1.3
Protected check	Sounding Line / Mainline Decision
Only RELEASE_GO emitter	scripts/sounding-line/finalizer.mjs through explicit sounding-line-authoritative.yml workflow_dispatch for exact frozen candidate SHA
Focused repair	Explicit diagnostic/qualification only; suite must exist in sealed plan; no RELEASE_GO
Protected binding	Consumes sealed artifacts and verifies identity/digests; cannot plan or execute product workers; cannot emit RELEASE_GO
Automatic broad closure	None in current workflow tree
Ordinary semantic carry-forward	Not implemented; fresh proof for all selected required nodes

2.2 Measured Worker-Cost Baseline

Representative successful authority run 31728114656 is the governing benchmark for v1.4 performance comparison unless Stage 1 records a newer controlled baseline before source implementation. Unavailable telemetry must remain unavailable rather than being backfilled with estimates.

Cost family	Sum	Average / maximum	Prompt 0 interpretation
npm ci	1,489 s	39.2 s / 79 s	Repeated on every worker; largest measured bootstrap family.
Checkout	1,154 s	30.4 s / 40 s	Repeated repository materialization is comparable to total suite execution.
setup-node	715 s	18.8 s / 48 s	Repeated runtime setup materially contributes to cumulative cost.
Suite execution	1,161 s	30.6 s / 379 s	Actual test work; browser.helm is longest at 379 s execution.

Declared preparation	177 s	4.7 s / 31 s	Resource-aware preparation already exists but is worker-local.
Whole authority run	1,180 s wall	5,042 worker-s / 84.0 worker-min	Strong intra-run parallelism but high cumulative repeated bootstrap.
Plan job	130 s total	npm ci 43 s; discovery 13 s	Separate setup path before worker fan-out.
Finalizer job	46 s total	finalizer command ~2 s	Already relatively light; not the primary throughput target.

2.3 Measurement Gaps

Prompt 0 could not reliably measure runner provisioning/queue latency, artifact-upload duration, teardown duration, artifact sizes, cache-hit state, or isolated database/browser sub-timings. v1.4 telemetry SHALL add these dimensions before claiming a complete performance model. Missing telemetry is not permission to infer a favorable result.

The underlying failure modes are distinct and must be measured separately:

- Overvalidation: a suite reruns even though no protected semantic dependency changed.
- Overprovisioning: a worker prepares dependency, database, browser, build, or provider resources not required by its sealed node.
- Tail serialization: candidate B cannot prove the exact tree expected after candidate A until A physically merges, even when that future tree can be deterministically synthesized.
- Binding repetition: heavy application proof is repeated at the final protected boundary even when source-bound evidence already proves the exact content tree.
- Coarse invalidation: broad path classes such as generic source or E2E directories expand scope beyond the actual contract dependency graph.
- Runner multiplication: one logical proof obligation maps to one freshly bootstrapped hosted machine even when compatible obligations can safely share an immutable prepared environment.

Not a coverage-reduction project

v1.4 treats redundant execution as a verification defect only when equivalence can be proven. When dependency identity is unknown, when a protected contract changed, or when a security/privacy/accessibility/migration obligation requires fresh environment proof, Sounding Line reruns it. Unknown means conservative proof, not optimistic reuse.

3. Goals, Non-Goals, and Success Criteria

3.1 Primary goals

1. Preserve and reuse authoritative evidence when its complete semantic dependency identity is unchanged.
2. Make ordinary mainline suite selection impact-based rather than a standing broad matrix.
3. Make prepared dependency, generated-client, browser, and certified database identities reusable across candidates when their complete identities match.
4. Reduce final protected binding to source/tree/policy/evidence/cleanup reconciliation unless binding itself discovers semantic drift.
5. Allow multiple qualified candidates to prove their predicted sequential integration trees concurrently while preserving one physical protected-main transition at a time.
6. Adopt usable evidence from paused v1.3 project branches rather than forcing a blanket retest after the authority upgrade.
7. Keep release-candidate assurance deep and exhaustive where release policy requires it.
8. Preserve fail-closed rollback and emergency serial operation throughout cutover.

3.2 Quantitative engineering objectives

The initial SLOs remain targets, but Prompt 0 establishes the baseline against which improvement SHALL be demonstrated. A target is not accepted merely because one fast candidate happens to meet it; the implementation must show proof-equivalent behavior across the governed shadow corpus.

Metric	Prompt 0 baseline	v1.4 initial objective	Governing interpretation
Ordinary mainline wall time	1,180 s representative run	5-15 min typical	Miss triggers optimization analysis, never evidence deletion.
Summed worker time	84.0 worker-min	Material reduction vs controlled baseline	Track cumulative runner cost as well as critical path.
npm ci	1,489 summed s	Eliminate repeated install on verified layer hits	Prepared layer must be identity-complete and benchmark-favorable.
Checkout + setup-node	1,869 summed s	Reduce through safe bundle/layer design	Never share mutable workspace state.
Protected finalizer	46 s job / ~2 s command	Binding <1-2 min total	Focus on tree/cleanup/provenance completeness.
Four-candidate train tail	Not implemented	<30-45 min target	Includes speculative qualification and ordered landing.
Conditional suite rationale	14 conditional; none selected; no impact explanation	Every select/omit decision sealed and explainable	Unknown mapping broadens scope.

Measure	Initial v1.4 objective	Interpretation
Warm ordinary mainline	5-15 min typical target	Target after prepared-layer reuse and semantic preservation are active. Not a gate escape hatch.
Protected binding	< 1-2 min after qualified evidence	Identity/provenance/tree reconciliation, not a second application test run.
Unrelated main advance	Majority of eligible proof preserved	Low preservation with no dependency change is an invalidation-quality incident.
Prepared artifact hit rate	>80% in steady multi-project swarm when identities match	Measure per artifact class; do not force reuse when transfer is slower or identity incomplete.
Four-candidate train tail	<30-45 min target	Includes speculative qualification and ordered landing; depends on actual test critical paths.
Release candidate	No weakened scope	Exhaustive release obligations remain governed independently of ordinary mainline speed.

3.3 Explicit non-goals

- Removing test tiers or negative cases because they are slow.
- Replacing journey-level evidence with unit evidence merely to save time.
- Changing One Voyage, Wayfarer, Harborlight, Sealed Hold, Helm, Drydock, or other product semantics.
- Building the future self-hosted/prewarmed worker pool as a mandatory v1.4 deliverable.
- Making mutable shared databases, ports, browser profiles, storage roots, or build outputs safe by assertion.
- Allowing a merge train to bypass the existing protected Mainline Decision.
- Automatically resolving semantic merge conflicts or migration collisions whose ownership is ambiguous.
- Using cache freshness, recent green status, or matching filenames as a substitute for complete identity.

4. Supersession Map and Cross-Part Boundaries

Version 1.4 is a cross-part amendment because its requirements span verification meaning, runtime mechanics, and repository/mainline enforcement. It does not collapse the three-document constitution. Each clause below is assigned to the authority domain that owns its meaning.

Existing authority area	v1.4 addition / clarification
Part I - Minimum sufficient evidence	Adds complete evidence-fingerprint closure, train-tree evidence composition, and binding equivalence requirements.
Part I - Semantic invalidation	Replaces broad-path steady-state behavior with contract/dependency closure and explicitly governs train/base movement.
Part I - Prepared Artifact Identity	Defines composable content-addressed layers and their immutable identity boundaries.
Part II - Scheduler / broker	Adds speculative train planning as a scheduling concern while preserving lease/resource authority.
Part II - workers	Adds prepared-layer restoration and compatible multi-suite worker bundling with independent receipts.
Part II - distributed runtime	Defines downstream train qualification and predicted-tree work that may run before upstream physical merge.
Part III - mainline	Introduces train admission, order, head-car ownership, replanning, and protected-tree binding.
Part III - cutover	Requires shadow mode, one atomic implementation cutover, and explicit rollback path.
Part III v1.3 workspace law	Prepared shared artifacts live outside phase worktrees; mutable run state remains owned and disposable.

5. Canonical v1.4 Vocabulary

Term	Definition
Semantic Evidence Fingerprint	Canonical digestable description of every production contract, source closure, test definition, fixture, schema, migration, toolchain, runtime, environment, policy, and resource identity on which a receipt's protected claim depends.
Protected Contract Closure	The transitive producer/consumer/source/test/fixture dependency set that can change the meaning of a verification obligation.
Prepared Artifact	Immutable, content-addressed execution input reused across workers or candidates, such as dependency environment, generated Prisma client, browser engine bundle, or certified SQLite baseline.
Prepared Layer	A prepared artifact designed to compose with other layers rather than containing the entire runtime universe.
Train	Ordered set of qualified or qualifying mainline candidates whose predicted integration states are derived sequentially from one base main state.
Train Car	One candidate in a train, with exact source head, expected parent state, predicted integration tree, required evidence set, and train position.
Predicted Integration Tree	Git tree object representing the exact repository contents expected if a train car were integrated using the governed merge strategy on its predicted parent state.
Actual Integrated Tree	Git tree of the protected main commit produced when a qualified car physically lands.
Tree-Bound Qualification	Evidence state in which the protected product contents are qualified against an exact Git tree independent of incidental commit metadata.
Protected Binding	Final authority operation that binds current source commit, parent/base/train identity, exact tree, policy, evidence, cleanup, and protected-check status without unnecessary re-execution.
Legacy Evidence Adoption	Auditable conversion/reconciliation of older v1.3 receipts into v1.4 dependency-aware evidence when sufficient provenance exists.
Train Brake	Fail-closed suspension of downstream promotion when predicted and actual state diverge or train assumptions become

Term	Definition
	invalid.
Train Replan Boundary	Earliest train car whose parent/source/policy/dependency assumptions changed; that car and downstream cars must be recomputed.
Head Car	Earliest not-yet-landed train car; the only car permitted to physically advance protected main.
Shadow Decision	Non-authoritative v1.4 plan/decision generated during cutover comparison. It may diagnose divergence but may not authorize merge.

6. Non-Negotiable Invariants

9. No mandatory obligation disappears because v1.4 is faster. Every gate still resolves each obligation to exactly one valid current producer.
10. Preservation requires complete auditable dependency equivalence. Unknown dependencies invalidate or trigger conservative fallback.
11. Prepared artifacts are immutable by contract. Writable run state is separately cloned or allocated.
12. One stable protected authority remains: Sounding Line / Mainline Decision.
13. Only the head train car may advance protected main. Speculation is permitted; unordered physical merges are not.
14. Predicted tree equality is necessary for downstream train evidence to survive an upstream merge. Tree mismatch brakes the train.
15. Commit SHA metadata differences may be tolerated only when the actual integrated tree is exactly the qualified predicted tree and final binding proves parent/source relationships.
16. Release-candidate evidence remains governed by release policy, not ordinary mainline throughput targets.
17. Security, privacy, authorization, accessibility, migration, cleanup, and source-binding vetoes are never averaged away by performance metrics.
18. All prepared-layer and train artifacts obey Part III v1.3 workspace, retention, and cleanup law.
19. Feature projects do not implement ad hoc train, preservation, or generic caching logic. v1.4 infrastructure remains Sounding Line-owned.
20. The v1.4 implementation itself cannot declare itself authoritative before the current authority accepts the atomic cutover candidate.

Decision rule

If v1.4 cannot prove that an optimization preserves the required claim, it does not optimize that claim. The correct fallback is narrower certainty when possible and broad fail-closed proof only when necessary.

7. Target Architecture Overview

The v1.4 control path separates planning, execution, and binding more aggressively than the pre-amendment path. The planner first computes the gate obligations and dependency fingerprints. The evidence store supplies current, preserved, rebound, or legacy-adoptable receipts. Only missing or invalidated obligations are scheduled. The scheduler composes prepared artifacts with run-owned mutable resources. Train planning may synthesize downstream integration trees while upstream authority work continues. The finalizer consumes all receipts and conformance evidence, while protected binding confirms that the candidate and actual integrated tree are the exact state the evidence described.

GATE REQUEST

- > resolve current Sounding Line authority
- > enumerate mandatory obligations
- > compute protected contract closure + dependency fingerprints
- > search current / preserved / rebound / adoptable evidence
- > schedule only unresolved obligations
- > restore prepared immutable layers

```

-> allocate run-owned mutable resources
-> execute / collect independent receipts
-> finalizer assembles MSES + runtime/workspace conformance
-> train/tree binding
-> Sounding Line / Mainline Decision

```

8. Semantic Evidence Fingerprint

Every receipt eligible for preservation SHALL expose a canonical semantic fingerprint. The fingerprint is not merely a list of changed paths. It is the auditable dependency identity for the claim. A browser receipt protecting Wakebook history, for example, may depend on Wakebook source, Wayfarer history projection, Tideglass handoff contract, selected fixtures, specific test definitions, schema semantics, application runtime, browser identity, and policy. An Admiralty-only change that touches none of those dependencies does not invalidate the receipt merely because both projects live under a common source directory.

Fingerprint family	Required contents
Claim identity	suite ID, test IDs/case set, protected contract IDs and versions, tier/risk floor, success semantics.
Production closure	canonical source tree entries and direct/transitive producer/consumer identities relevant to the protected contracts.
Persistence	schema digest, migration range/digest, generated-client identity, baseline/seed semantics when applicable.
Test definition	test source digest, registration metadata, shared assertion/helper libraries, fixture builders, expected case inventory.
Dependencies/toolchain	package lock, Node/npm, compiler, Prisma, Playwright/browser revision, build adapters, native/OS constraints as required.
Environment	environment class only when the protected claim depends on it; provider identity where live/provider proof is part of the claim.
Policy	Sounding Line authority index, gate policy, risk floor, resource contract, finalizer semantics, planner policy digest.
Execution conformance	prepared artifact identities, run-owned resources, isolation/cleanup receipt, worker/boot identity when relevant.

8.1 Canonicalization rules

- Fingerprint serialization must be deterministic and versioned.
- Path order, set order, JSON key order, and platform-normalized path representation must not create accidental identity drift.
- Hash algorithms and canonicalization version are machine-readable policy.
- A fingerprint references immutable digests rather than embedding secrets or large artifacts.
- Fingerprints must distinguish semantic source closure from incidental repository files that cannot affect the claim.
- A collision or canonicalization ambiguity is an evidence-integrity failure, not a preservation opportunity.

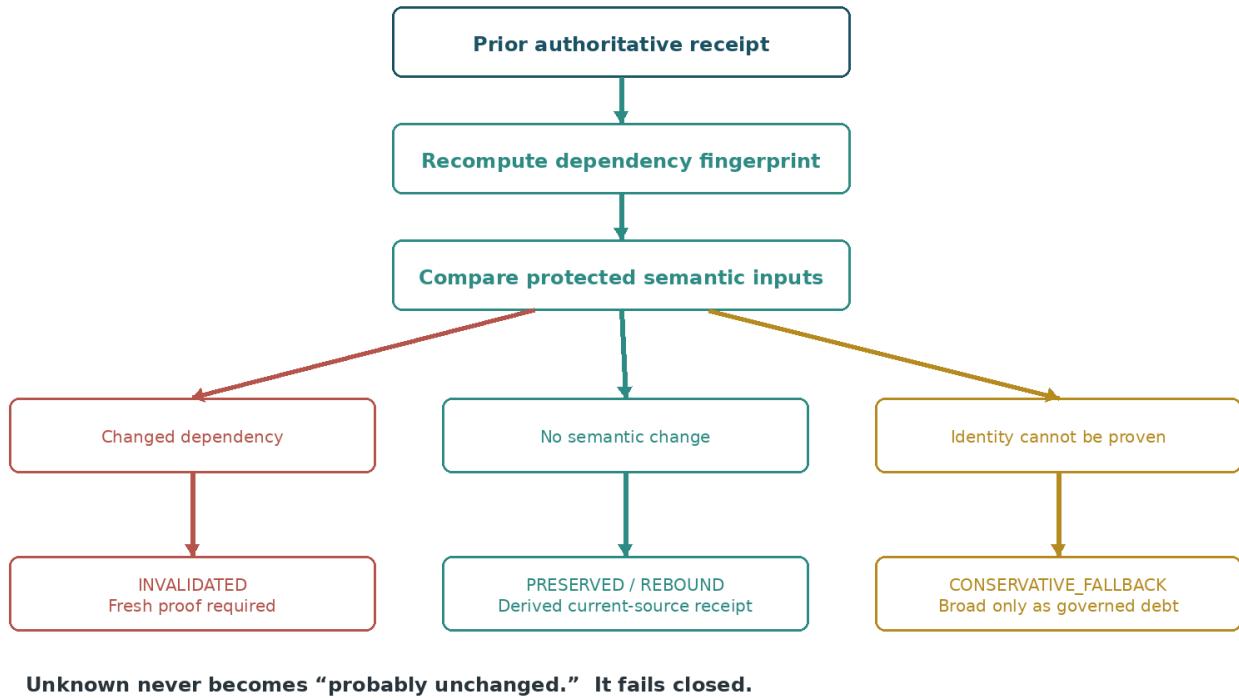
Figure 2. Semantic evidence preservation decision

Figure 2. v1.4 preservation is a dependency-equivalence decision, not a SHA-equality shortcut.

9. Contract Closure and Transitive Dependency Resolution

The invalidation engine must be able to explain why a source, schema, fixture, tool, or policy input belongs to a receipt. Contract closure begins with protected contract IDs, then traverses registered producers, consumers, adapters, projections, test helpers, fixtures, and generated artifacts. Static import graphs may help but are not sufficient by themselves; dynamic routes, registry lookups, schema consumers, code generation, environment contracts, and policy mappings must be represented in the machine-readable impact map.

9.1 Closure quality classes

Class	Meaning	Planner behavior
EXACT	Every relevant producer/consumer and semantic dependency is represented and versioned.	Preservation eligible.
BOUNDED	Some implementation detail is conservatively grouped but the bound is explicit and small.	Preservation eligible within recorded bound.
COARSE	Broad directory/category is used because precise mapping is incomplete.	May preserve outside bound; record mapping debt.
UNKNOWN	Sounding Line cannot prove what can affect the obligation.	Fresh proof or conservative fallback; never preserve optimistically.

9.2 Inverse impact is mandatory

A dependency closure must support both directions: changed input -> affected obligations and obligation -> complete relevant input closure. The inverse form is necessary to prove preservation.

9.3 Prompt 0 Mapping-Hardening Gate

Impact minimization is prohibited wherever Sounding Line cannot prove sufficient mapping closure. Prompt 0 found 475 contracts, 16 owners, 156 path mappings, 448 contract mappings, 61 suites, 19 resources, and 27 unmapped contracts. This creates a mandatory hardening workstream inside Stage 2. UNKNOWN and INSUFFICIENT areas SHALL conservatively expand proof until their mappings are repaired.

Area	Prompt 0 class	Coverage / evidence	v1.4 rule
Homeport, Admiralty, Bridgewatch, Wakebook, Tideglass	READY	100% current contract-to-suite mapping	Eligible for fine-grained minimization after fingerprint proof.
Wayfarer / Harborlight / Helm	PARTIAL	93% / 90% / 89%; critical gaps remain	Close critical gaps or broaden affected obligations.
Platform foundation / One Voyage / Lanternwake / Sealed Hold	PARTIAL	Approx. 29%-50%; broad owner paths	Use conservative floors until closure is complete.
Drydock / Deepwater / Landfall	INSUFFICIENT	0% contract-map coverage despite critical contracts	No omission based on contract map until hardened.
Breakwater	UNKNOWN	Owner paths/suites exist but no contracts	Absence is not safety; treat as unknown impact.

When a change occurs, Sounding Line SHALL answer both directions: which obligations does the change invalidate, and for every preserved obligation, why is the changed interval outside that obligation's protected closure? This makes omission reasoning inspectable and prevents a "nothing matched the path glob" implementation from becoming false assurance.

10. Evidence Preservation, Rebinding, and Invalidation

v1.4 retains the v1.2 disposition vocabulary. "Carried-forward" is a human description of a PRESERVED derivation, not a competing status. Historical evidence remains immutable. The current candidate receives a new derivation record that explains why the prior proof still satisfies the present obligation.

Disposition	v1.4 meaning	Action
FRESH	Receipt executed on the exact required source/tree/policy/environment identity.	Consume directly.
PRESERVED	Prior proof remains semantically unchanged; current derivation proves fingerprint equivalence.	Consume derived receipt.
REBOUND	Claim semantics unchanged but source-bound metadata/record must be regenerated.	Regenerate and validate record.
INVALIDATED	One or more protected dependencies changed.	Schedule fresh proof.
SUPERSEDED	Newer or stronger current evidence replaces older evidence.	Retain history; count once.
CONSERVATIVE_FALLBACK	Safe preservation cannot be proven because mapping/identity is incomplete.	Rerun governed scope and create implementation debt.

10.1 Main-advance rule

When main advances under a queued candidate, the candidate SHALL reconcile the changed interval and classify every affected mandatory obligation. Unrelated source movement does not imply full invalidation. Shared contract, schema, harness, policy, or generated-artifact movement may broaden the scope through the registered dependency graph. Exact integration/tree proof remains fresh for the current base.

10.2 Preservation expiry

- Protected contract semantics changed.
- Test/fixture/assertion definition changed in a way that can alter the claim.
- Schema/migration/persistence semantics changed for a database-dependent claim.
- Toolchain/browser/provider identity changed for an environment-sensitive claim.
- Authority/gate/risk/finalizer semantics changed incompatibly.
- Prepared artifact identity is missing, mismatched, mutable, or untrusted.
- The original receipt or derivation chain cannot be integrity-verified.

11. Legacy v1.3 Evidence Adoption

The authority cutover must not greet paused project branches by discarding every focused receipt simply because v1.4 introduced richer metadata. Legacy adoption is an evidence migration, not a trust shortcut. An older receipt can contribute only when Sounding Line can reconstruct enough of its protected claim, source identity, test definition, fixture/persistence context, environment, and policy to build a valid v1.4 dependency fingerprint.

Legacy state	Adoption disposition
Complete reconstructable provenance	Create v1.4 adoption/derivation record; evidence may become PRESERVED or REBOUND.
Claim known, some non-semantic metadata missing	Regenerate missing metadata and rebind if equivalence can be proven.
Dependency closure incomplete but bounded	Rerun only the obligations whose equivalence cannot be established.
Test result present but source/test/fixture identity ambiguous	Historical only; fresh proof required.
Legacy result failed, flaky, cancelled, blocked, or cleanup-invalid	Cannot be promoted by adoption. Resolve under current failure policy.

11.1 Adoption receipt

```
{
  "legacyReceiptDigest": "...",
  "legacyAuthority": "Sounding Line v1.3",
  "priorSourceSha": "...",
  "reconstructedDependencies": [...],
  "missingDependencies": [],
  "disposition": "PRESERVED | REBOUND | INVALIDATED",
  "reasonCode": "...",
  "newSourceSha": "...",
  "plannerPolicyDigest": "..."
}
```

12. Impact-Selected Ordinary Mainline Gate

The ordinary mainline gate SHALL no longer have a steady-state architecture in which every project phase automatically executes a large fixed matrix simply because the matrix exists. Instead, the planner starts with mandatory global sentinels, then adds semantically affected product suites, invalidated prior obligations, affected cross-project contracts, missing evidence, and exact integration/tree proof.

```
ORDINARY MAINLINE MSES
= always-fresh sentinel obligations
+ semantically affected suites/contracts
+ obligations invalidated by base movement
+ cross-project boundary evidence selected by dependency closure
+ missing mandatory gate evidence
+ runtime/workspace conformance
+ exact candidate/train/tree binding evidence
```

12.1 Planner omission record

For every normally eligible suite not selected, the sealed plan SHALL record the reason it is unnecessary for this gate. This requirement is intentionally symmetrical with execution selection: v1.4 must explain both why it runs something and why it safely does not rerun something.

12.2 Broad fallback

A broad fallback is a conservative proof mechanism, not the normal steady state. Its activation SHALL be recorded with cause, affected unknown mapping, and debt owner.

12.3 Conditional-Suite Activation Is a Real Decision

Prompt 0 found fourteen conditional mainline suites that are currently omitted because ordinary planner input does not contain changed-path/contract impact, not because the planner proved them irrelevant. v1.4 SHALL replace this accidental omission with explicit selection logic. Every conditional suite SHALL receive one sealed disposition: `SELECTED_BY_IMPACT`, `SELECTED_BY_RISK_FLOOR`, `PRESERVED_BY_FINGERPRINT`, `OMITTED_WITH_PROOF`, or `CONSERVATIVE_FALLBACK_REQUIRED`. A missing rationale invalidates the plan.

Prompt 0 conditional inventory: `migration.mysql`; `browser.auth`; `browser.invitations`; `browser.player-library`; `browser.player-journal`; `browser.passport`; `browser.artifacts`; `browser.captain`; `browser.studio`; `browser.community`; `browser.private-operations`; `browser.navigation`; `browser.responsive`; `browser.animation-lifecycle`.

A broad mainline fallback remains legal when the planner cannot prove a smaller correct set. It must be labeled `CONSERVATIVE_FALLBACK`, name the missing dependency/invalidation capability, create Sounding Line debt, and remain visible in throughput metrics. It is not allowed to become the unexamined default.

13. Always-Fresh Sentinel Spine

Some obligations exist to catch broad integration failure classes and should normally execute fresh on ordinary executable mainline candidates. The exact sentinel registry is machine-readable and may evolve, but its purpose is stable: catch repository-wide breakage that semantic preservation alone should not hide.

Sentinel family	Default v1.4 expectation	Why fresh
Sounding Line conformance/self-tests	Always fresh when executable source changes or authority runtime changes.	Proves the planner/runtime/finalizer that reused evidence are themselves valid.
Core static/type/policy sentinel	Fresh according to source/gate policy.	Cheap broad repository contract and policy safety.
Access/navigation sentinel	Fresh when user-facing executable source can affect global access.	Catches catastrophic shell/auth reachability failures

Sentinel family	Default v1.4 expectation	Why fresh
		across project boundaries.
Production build	Normally fresh for executable/build-affecting source; may be policy-scoped for record-only changes.	Build graph can fail outside local test closures.

Sentinel rule

A sentinel is not a backdoor full matrix. Sentinel status must remain intentionally small, stable, high-value, and separately justified. If every suite is declared a sentinel, the policy is nonconformant.

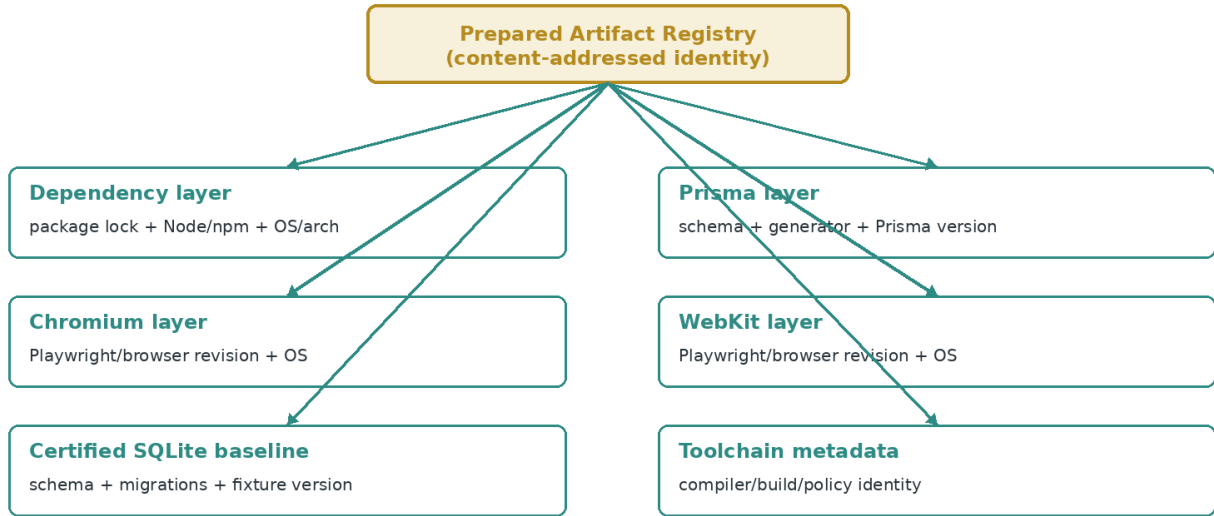
14. Release-Candidate Separation

The release-candidate gate remains the deep pre-release assurance surface. v1.4 optimizes ordinary phase integration and evidence reuse; it does not redefine release coverage. Release-candidate policy may intentionally execute evidence that ordinary mainline preserved or did not require, including provider, device, compatibility, owner/manual, recovery, and complete current-product obligations.

Gate	v1.4 ordinary behavior
local-change	Impact-selected fast feedback; never merge-authoritative.
subsystem	Complete project/subsystem closure proof; supports candidate qualification.
mainline	Minimum sufficient current evidence + fresh sentinels + exact integration/tree + conformance/cleanup.
record-closure	Narrow record/policy/document integrity when executable semantics unchanged.
release-candidate	Deep release obligations; no scope reduction merely because mainline was green or train-qualified.

15. Prepared Immutable Runtime Layers

v1.4 adopts Option B as the primary prepared-runtime architecture: content-addressed immutable layers that can be shared safely across candidates and workers. The identity is based on the inputs that determine the artifact, not on the candidate that happened to request it. If eight candidates use the same package lock, Node/npm identity, OS/architecture, Prisma schema, and browser versions, the corresponding prepared layers should normally be reused rather than rebuilt eight times.

Figure 3. Composable prepared immutable runtime layers

Mutable run state is never shared: databases, browser contexts, ports, storage roots and processes remain run-owned.

The architecture governs identity and immutability, not a particular archive format. Transport must be benchmarked.

Figure 3. Prepared runtime identity is composable. Mutable run state is deliberately outside the shared layers.

15.1 Required layer classes

Layer	Canonical identity inputs	Mutable state excluded
Base toolchain metadata	OS, architecture, Node/npm, execution policy, adapter version.	Candidate worktree, DB, browser context, ports.
Dependency layer	package.json/lock, Node/npm, OS/arch, install-policy digest, native dependency class.	Project-specific generated state unless declared.
Prisma generated-client layer	dependency identity, Prisma version, schema digest, generator configuration.	Writable database.
Chromium layer	Playwright version, browser revision, OS/arch, browser policy.	Profile/context/cache writes for a run.
WebKit layer	Playwright version, browser revision, OS/arch, browser policy.	Profile/context/cache writes for a run.
Certified SQLite baseline	schema, migration history, deterministic seed/fixture version, baseline certification policy.	Per-test mutations; each mutating consumer receives clone/copy.

15.2 What v1.4 explicitly rejects

The prepared-layer model is not permission to share mutable node_modules workspaces, browser profiles, writable SQLite files, ports, storage roots, or build outputs between runs. Those remain run-owned resources.

15.3 Prompt 0 Prepared-Layer Feasibility

Prompt 0 independently assessed all five proposed layer classes as feasible on hosted and future self-hosted workers when they are immutable, content-addressed, verified, and consumed as identity-bearing inputs. Current size and isolated sub-timing measurements are incomplete and SHALL be benchmarked before transport is frozen.

Layer	Prompt 0 measured/current evidence	Required identity / invalidation	v1.4 disposition
-------	------------------------------------	----------------------------------	------------------

Dependency/runtime	npm ci avg 39.2 s per worker	package-lock, Node/npm, OS/arch, install flags, package integrity	Highest-priority layer; benchmark restore vs warm npm ci.
Prisma client	18 current consumers; isolated cost unavailable	schema, generator config, Prisma version, dependency layer, Node/OS/arch	Manifest-hashed generated layer or regenerate-and-compare.
Chromium	4 current consumers; prep max 31 s includes other work	Playwright package/revision, OS/arch, content digest	Exact revision layer; tamper/revision mismatch rebuilds.
WebKit	0 current mainline consumers; unmeasured	same class as Chromium	Support protocol now; measure when used.
Certified SQLite baseline	5 migration/seed consumers; isolated timing unavailable	schema, migrations, seed, Prisma client, SQLite, baseline manifest	Immutable baseline; each mutating test gets run clone.

- A generic mutable node_modules directory treated as authoritative merely because it exists.
- One giant candidate-specific runtime archive containing every browser/database/tool whether required or not.
- Restoring a prepared layer whose identity cannot be verified against the sealed plan.
- Sharing mutable database or browser state inside an immutable layer.
- Embedding real credentials, private Chronicle content, production data, or unbounded test artifacts in reusable layers.

16. Prepared Artifact Identity and Security

Prepared artifacts are evidence-bearing supply-chain objects. Their convenience makes them security-sensitive: a poisoned dependency layer can affect many workers, so v1.4 requires stronger identity and provenance than an ordinary opportunistic cache.

Field	Requirement
artifactType	dependency prisma-client browser-chromium browser-webkit sqlite-baseline approved future type
identityVersion	Canonicalization/schema version.
inputDigests	All complete identity inputs and their digests.
artifactDigest	Digest of the immutable produced artifact or deterministic manifest.
producer	Trusted workflow/worker identity and boot/runner class where required.
createdAt	Auditable creation time.
policyDigest	Prepared-artifact policy controlling identity and eligibility.
securityScan	Required malware/private-content/supply-chain status appropriate to class.
retentionClass	Shared immutable cache policy; not phase worktree retention.
consumerConstraints	OS/arch/runtime/browser/native constraints.

16.1 Trust boundary

Untrusted pull requests or feature branches may request or consume trusted prepared artifacts but may not silently publish a globally trusted layer unless the repository policy explicitly authorizes the producer path. Layer publication must prevent cache poisoning, key collision, artifact substitution, and mutable post-publication edits.

17. Transport, Storage, and Lifecycle of Prepared Layers

The architecture mandates content-addressed identity and immutability, not a particular compression format, artifact service, or GitHub cache trick. Stage 4 must benchmark at least the viable transport strategies for the current hosted environment. Restoring a huge archive that takes longer than a warm dependency install is not an optimization; it is npm ci wearing luggage.

17.1 Transport decision inputs

- Artifact size and compression cost.
- Restore/download bandwidth and hosted cache behavior.
- Hit rate across the active project swarm.
- Native file count and filesystem extraction overhead.
- Integrity verification time.
- Storage quota and eviction behavior.
- Cross-OS portability requirements.
- Security and trust of producer/consumer contexts.

17.2 Lifecycle

Prepared artifacts SHOULD live in a shared immutable cache/artifact area outside phase worktrees, consistent with Part III v1.3. They may outlive an individual run but remain bounded by retention and storage policy. Their registry records usage, last verification, size, hit/miss rates, and reason for rebuild or eviction. An artifact is rebuildable; its identity record and evidence history are durable only as required by governance.

18. Worker Bundling and Per-Suite Receipt Boundaries

v1.4 may reduce hosted job count by assigning multiple compatible suites to one physical worker, but logical suite identity and evidence boundaries remain exact. Bundling is a scheduling optimization, not a test-accounting shortcut.

Bundling invariant	Requirement
Compatibility	Suites share safe immutable layers and have non-conflicting mutable resource requirements.
Isolation	Each suite receives its declared run-owned DB/storage/browser/process state.
Receipts	Each logical suite emits independent case counts, result, timing, artifacts, cleanup, and protected-claim identity.
Failure	One suite failure does not automatically convert compatible sibling suites into failures; dependencies determine blocked/cancelled states.
Cancellation	Planner may stop remaining bundle nodes according to dependency/cost policy while preserving completed receipts.
Conformance	BUNDLE_RECEIPT_BOUNDARY_INVALID remains a hard failure when independent receipt completeness cannot be proven.

19. Git Source Identity and Tree Identity

Git commit identity includes metadata and parent relationships in addition to file contents. A speculative train may qualify a predicted integration commit before GitHub creates the physical merge commit, so v1.4 distinguishes commit SHA from tree SHA without weakening exact-source binding.

19.1 Identity layers

Identity	Meaning	Use
Candidate head SHA	Exact feature source commit.	Proves which candidate content is proposed.
Predicted parent commit/tree	Train state on which the candidate is synthetically integrated.	Defines the expected integration context.
Predicted integration tree SHA	Exact file-tree contents qualified by downstream evidence.	Content equivalence anchor.
Actual protected-main commit SHA	Physical commit produced by governed merge.	Final repository/history identity.
Actual integrated tree SHA	Tree contained by actual protected-main commit.	Must match qualified predicted tree for downstream preservation.

19.2 Tree equality is necessary, not sufficient

Matching tree content proves the tested repository bytes survived the physical merge. Protected binding must still verify candidate identity, expected parentage/base, merge strategy, policy, evidence set, cleanup, and protected branch conditions. Tree equality does not excuse an unauthorized source substitution or wrong parent history.

20. Ultra-Light Protected Binding

Once a candidate or train car has a complete MSES against its exact predicted integration tree, the protected binding operation should not repeat application tests simply because it is the final step. Binding is primarily provenance and accounting.

```

PROTECTED BINDING
  verify candidate head SHA
  verify expected train/base identity
  verify actual/predicted integration tree equality
  verify governed merge strategy + parentage
  verify policy / planner / finalizer digests
  verify MSES completeness
  verify preserved/rebound provenance chains
  verify prepared-artifact identities consumed by fresh proof
  verify runtime + workspace cleanup conformance
  verify no train/source/policy drift
  -> emit existing Sounding Line / Mainline Decision

```

20.1 Current Binding Is Already Evidence-Only; v1.4 Finishes Identity Proof

Prompt 0 verified that ordinary protected binding already consumes sealed artifacts and does not rerun product suites. The representative finalizer job was 46 seconds total, with the finalizer command itself near 2 seconds. Therefore v1.4 SHALL not claim major savings by removing tests from binding that are already absent. Its binding work is to add predicted/actual tree equality, sealed cleanup-resource provenance, authority-version coherence, and to reduce avoidable checkout/Node/bootstrap where practical. Record-only binding still performs npm ci and is a secondary optimization opportunity.

20.2 Binding Completion Conditions

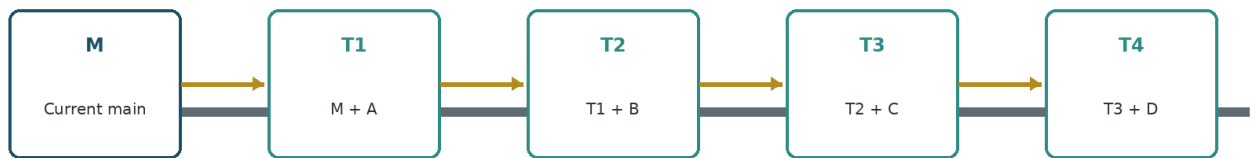
A binding is complete only when candidate/base/train identity, expected parentage, predicted tree, actual tree, sealed plan/finalization/evidence digests, current authority digest, cleanup-resource disposition, and branch-protection status reconcile. Tree equality proves byte identity but does not excuse wrong parent history, candidate substitution, or evidence from a different authority version.

21. Mainline Train Model

Prompt 0 classifies merge-train feasibility as PARTIAL FOUNDATION, NOT READY. Existing GitHub synthetic merge and protected binding prove parent composition only. No current component builds ordered candidate trees, seals predicted tree SHAs, manages queue/withdrawal, replans after head-car failure, handles external main advancement, verifies GitHub merge-tree equality, or binds migration order to train state. v1.4 SHALL implement and prove those functions before train execution can become authoritative.

The Mainline Train is an ordered sequence of candidates qualified against deterministic future integration states. It preserves one protected main history while eliminating dead queue time between candidate finalizations.

Figure 4. Speculative ordered mainline train



Only the head car may physically advance protected main.

Downstream cars may qualify their predicted integration trees concurrently.

After each merge: actual integrated tree == qualified predicted tree -> downstream proof survives.

Mismatch, mutation, conflict, policy drift or external main advance -> brakes + replan from first affected car.

Figure 4. Downstream train cars can qualify predicted sequential trees while only the head car owns the physical protected-main transition.

21.1 Canonical construction

```

M = current protected main
A = first candidate
B = second candidate
C = third candidate
D = fourth candidate

```

```

T1 = governed_synthetic_integrate(M, A)
T2 = governed_synthetic_integrate(T1, B)
T3 = governed_synthetic_integrate(T2, C)
T4 = governed_synthetic_integrate(T3, D)

```

The synthetic integration algorithm must match the repository's configured merge strategy closely enough that the expected tree is deterministic and comparable with the later physical merge. If the repository changes merge strategy, train policy and affected evidence are invalidated.

22. Train State Machine and Admission

State	Meaning	Allowed transitions
DISCOVERED	Candidate is visible but not yet train-eligible.	-> QUALIFYING or WITHDRAWN
QUALIFYING	Subsystem evidence and current source reconciliation are in progress.	-> READY or FAILED/STALE
READY	Candidate is frozen and can be ordered into a train.	-> TRAINED or WITHDRAWN
TRAINED	Position/base/predicted tree assigned.	-> SPECULATIVE_VALIDATION
SPECULATIVE_VALIDATION	Evidence executing/closing against predicted tree.	-> TRAIN_GREEN or TRAIN_BRAKED
TRAIN_GREEN	All mandatory evidence for predicted tree is complete.	Head -> BINDING; downstream waits for upstream tree confirmation.
BINDING	Head car is being bound/merged to protected main.	-> LANDED or TRAIN_BRAKED
LANDED	Actual main tree matches qualified tree and main advanced.	Remove car; confirm next car.
TRAIN_BRAKED	Assumption failed; downstream promotion suspended.	-> REPLAN or FAILED/WITHDRAWN

22.1 Admission requirements

- Candidate source head is frozen and pushed.
- Required subsystem qualification is complete or explicitly part of train evidence according to policy.
- Candidate has no unresolved known product/test/security/privacy/accessibility defect.
- Synthetic integration into predicted parent succeeds without unresolved conflict.
- Migration/schema ownership checks pass.
- Required train metadata and cleanup state exist.
- Candidate has not begun a later same-project phase that would stack unaccepted work.

23. Candidate Mutation, Withdrawal, and Replanning

Train performance comes from limiting the blast radius of change. Replanning begins at the earliest car whose assumptions changed, not automatically at car 1.

Event	Required replan boundary
Candidate C receives a new commit	C and all downstream cars. A/B remain valid if unchanged.
Candidate B is withdrawn	Remove B; recompute C onward against the new parent state.
Head A fails product evidence	A leaves/pauses; recompute train from the new first car/base.
Protected main advances externally	Suspend promotion; reconcile new base and recompute from first dependency-affected car, preserving unaffected evidence.
Train policy/finalizer semantics change	Invalidate every car whose decision semantics can change; often whole train.
Prepared artifact revoked/tampered	Invalidate every fresh receipt depending on that artifact; re-execute affected obligations.

Event	Required replan boundary
Actual landed tree != predicted tree	Immediate train brake; next car and all downstream trees are invalid until replan.

24. Merge Conflicts, Migrations, and Shared Schema Changes

The train does not “solve” semantic conflicts. It detects them earlier. A candidate that cannot be deterministically integrated into its predicted parent is not green train cargo.

24.1 Source conflicts

Textual conflicts require project/contract ownership resolution before the car can proceed. The train planner records conflict paths and owner contracts but does not auto-prefer head, base, or timestamp.

24.2 Migration collisions

Database migration identifiers, order, checksums, forward/backward compatibility, and schema-family parity are shared integration dependencies. A collision is an integration defect, not something to rename blindly. Train planning must detect occupied ranges/names and require a governed resolution before downstream database evidence is considered valid.

24.3 Shared schema change

A schema or migration change can invalidate multiple downstream cars even if their source paths are unrelated. The dependency graph must represent schema consumers, generated clients, baseline identity, and fixtures so invalidation broadens for a semantic reason rather than for the mere presence of ``prisma/**``.

25. Fairness, Priority, and Starvation Prevention

v1.4 adds speculative concurrency but must not create a queue where large candidates permanently starve smaller or lower-priority work, or where a stream of tiny record updates constantly reorders executable candidates.

Policy	Rule
Default order	Qualified-ready timestamp with fairness aging.
Same-project sequence	A later phase cannot train ahead of its unaccepted prior phase.
Dependency priority	A candidate that unblocks multiple ready dependents may receive bounded priority boost.
Security/emergency	Explicit emergency policy may preempt normal ordering; preemption is audited and causes deterministic downstream replan.
Record-only work	Use record-closure or approved narrow lane; do not insert paperwork into the executable train when it does not need the train.
No chat diplomacy	Train order is machine-readable. Manual reorder requires recorded reason/authority.

26. Security Threat Model

v1.4 increases reuse and speculative execution, so it expands the trust surface. The security model must assume that a malicious or buggy candidate may attempt to influence shared caches, evidence identity, train order, or downstream proof.

Threat	Required control
Stale receipt replay	Receipt digest, source/policy/environment identity, derivation chain, and expiry checked by finalizer.
Evidence substitution	Protected obligation IDs and receipt digests sealed into plan/finalizer record.
Dependency-map poisoning	Policy-reviewed ownership/impact registries; conformance tests and conservative fallback on unknowns.
Prepared-layer cache poisoning	Trusted producer policy, immutable digest verification, consumer constraints, post-create immutability.
Hash/canonicalization collision	Versioned canonicalization and approved strong digest; mismatch/ambiguity hard fail.
Cross-OS native mismatch	OS/arch/native ABI included in identity where applicable.
Candidate substitution	Exact head SHA and train car identity bound before and after execution.
Train reorder	Order/position/base/tree sealed; unauthorized reorder invalidates affected cars.
Downstream reuse after drift	Actual/predicted tree equality gate and train brake.
Secret leakage into layer	Artifact-class scan and manifest; no credentials/private data; redaction and retention rules.
Mutable “immutable” artifact	Read-only publication, digest verification on consume, mutation incident on mismatch.
Untrusted PR publishes shared artifact	Producer trust policy separates consumer privilege from publisher privilege.

27. Privacy, Secrets, and Supply-Chain Boundaries

Prepared artifacts must contain only material safe for the artifact’s trust and retention class. A dependency or browser layer should contain no real Chronicle content. A certified database baseline should use synthetic governed fixtures unless a separately authorized evidence class explicitly requires protected data, in which case that baseline is not a generally reusable shared artifact.

- Never embed GitHub tokens, provider keys, database credentials, session cookies, private package keys, or production secrets in prepared artifacts.
- Never promote a run-owned database containing user data into the shared baseline registry.
- Private-content and secret-scanning gates apply to layer manifests and publication paths where appropriate.
- Artifact logs and manifests identify digests and safe metadata, not secret values.
- Supply-chain changes such as lockfile, runtime, package manager, browser revision, Prisma version, compiler, or generation adapter participate in identity when claims depend on them.

28. Throughput Metrics and Performance SLOs

Prompt 0 provides the first governed quantitative baseline for v1.4: authority run 31728114656, 1,180 seconds wall-clock, 38 workers, 5,042 summed worker-seconds, and a bootstrap-heavy cost profile. These figures are design evidence, not permanent constants; a Stage 7 performance report SHALL compare the same metrics under controlled v1.4 scenarios.

28.2 Required Prompt 0 Baseline Dimensions

Dimension	Prompt 0 value	v1.4 requirement
-----------	----------------	------------------

Wall-clock critical path	1,180 s	Record same gate/corpus before and after.
Summed worker time	5,042 s / 84.0 worker-min	Track cumulative cost in addition to wall time.
npm ci	1,489 s	Report hits/misses, transfer/build time, bytes, and saved setup.
Checkout	1,154 s	Report bundle/workspace optimization separately.
setup-node	715 s	Track runtime-layer hit/miss and bundling.
Suite execution	1,161 s	Do not hide genuine test cost under CI overhead.
Declared preparation	177 s	Split Prisma/DB/browser preparation where telemetry permits.
Unavailable today	queue/provisioning, artifact upload, teardown, artifact size, cache hits, isolated DB/browser timings	Instrument or explicitly continue marking unavailable.

Performance is measured to expose control-plane waste, never to justify weaker proof. v1.4 extends v1.2 telemetry so train throughput can be attributed to the correct layer.

Metric	Meaning
Queue time	Hosted runner and Sounding Line resource waiting, separated from product execution.
Prepared-layer restore time	Time to locate, transfer, verify, and materialize a reusable layer.
Prepared-layer hit/miss	Per layer class, including rebuild reason.
Fresh obligation count	How many mainline obligations required new execution.
Preserved/rebound count	How many obligations were safely satisfied without fresh product execution.
Over-invalidation ratio	Fresh execution with no changed semantic dependency. Target trends to zero.
Setup-to-execution ratio	Detects worker classes dominated by bootstrap.
Physical workers / logical suites	Measures bundling efficiency while preserving receipt boundaries.
Train depth	Number of candidate cars qualified concurrently.
Train tail latency	Time from first car acceptance start to last eligible car landing.
Train replan count/cause	Mutation, conflict, external main, policy drift, tree mismatch, withdrawal.
Predicted-tree match rate	How often physical merge tree exactly matches qualified tree.
Protected binding duration	Final identity/provenance binding time.
Closure latency	CLOSURE_FROZEN to merge-ready and to landed main.
Mapping hardening	Critical unmapped contracts are closed or explicitly classified as conservative proof debt; Drydock/Deepwater/Landfall cannot receive fine-grained omission while coverage remains insufficient.
Sealed-plan semantic completeness	Plan seals owner, contracts, command/adaptor identity, duration/budget, conditional rationale, resource identity and all digests used to omit or preserve proof.
Conditional-suite explanation	All 14 Prompt 0 conditional families receive machine-readable select/preserve/omit/fallback disposition for representative impacted candidates.
Cleanup provenance	CLEAN becomes a verified resource-disposition manifest, not only an asserted receipt field.

28.1 SLOs are diagnostics, not escape hatches

Missing a 5-15 minute ordinary-mainline target creates an infrastructure/performance investigation. It never authorizes deleting a mandatory obligation. Persistent misses should be decomposed into queue, restore/setup, suite critical path, teardown, finalizer, train replan, and binding. The remediation belongs to the dominant layer.

29. Part III v1.3 Workspace-Lifecycle Compatibility

v1.4 MUST preserve the v1.3 rule that local development state is temporary and unique work is preserved. Prepared immutable artifacts are specifically the kind of safe shared read-mostly cache v1.3 anticipated, but they should not cause every phase worktree to remain hydrated.

v1.3 rule	v1.4 application
Dormant worktrees source-only	Prepared layers live outside dormant worktrees; consumers restore/mount only when active.
Successful run state transient	Per-run DBs, browser contexts, traces, storage roots, ports and processes are cleaned after evidence sealing.
Shared read-mostly caches encouraged	Prepared artifact registry/cache is the governed shared mechanism.
Heavy worktree budget	v1.4 reduces repeated per-worktree hydration rather than raising the budget.
Workspace Cleanup Receipt	Remains mandatory; train eligibility and mainline finalizer consume cleanup/resource disposition where policy requires.
Disk-state safety	Layer retention/eviction participates in workspace capacity policy; cache cannot consume the machine into STOP_HEAVY_EXECUTION.
testing/evidence-lineage / adoption equivalent	Immutable derivation chain, original receipt references, changed-interval analysis, compatibility-by-authority-version, adoption/rebound rationale.
sealed plan schema	Must include/digest suite owner, protected contracts, command/adaptor, duration/budget, conditional rationale, dependency fingerprint inputs, prepared-layer requirements and mutable resources.
cleanup/disposition schema	Run-owned database/browser/port/process/storage/build identities, ownership, cleanup action, terminal state and unresolved-resource fail-closed behavior.

30. Shadow Mode and Decision Equivalence

v1.4 modifies the authority machinery that will later judge every project, including itself. Therefore the current pre-v1.4 authority remains authoritative while v1.4 is developed. v1.4 may run planners, preservation logic, prepared-layer execution, train simulations, and finalizer comparisons in SHADOW mode, but a shadow RELEASE_GO has no merge authority.

30.1 Required shadow corpus

- Representative recent ordinary project candidates.
- Known unrelated-main advances.
- Test-only repair.
- Record-only closure.
- Shared schema/migration change.
- Package-lock/toolchain change.
- Security/privacy/auth contract change.
- Sounding Line runtime/policy change.
- Intentional mapping UNKNOWN scenario.
- Prepared layer identity mismatch/tamper.
- Train candidate mutation and withdrawal.
- Predicted/actual tree mismatch injection.

30.2 Equivalence criterion

For every scenario, compare mandatory obligation sets and decision claims against the current correct authority. v1.4 may execute fewer redundant suites, schedule differently, preserve evidence, or report root/cascade states more precisely. It may not omit a mandatory protected claim or emit RELEASE_GO when the current authority would expose a genuine missing or failed obligation.

31. Atomic Authority Cutover

The v1.4 implementation is intentionally one atomic cutover candidate rather than seven separately merged authority fragments. Partial deployment would create a dangerous mixed state in which one component assumes semantic preservation while another still requires broad invalidation, or a train planner exists without a tree-aware binder.

Figure 5. v1.4 shadow-mode and atomic authority cutover

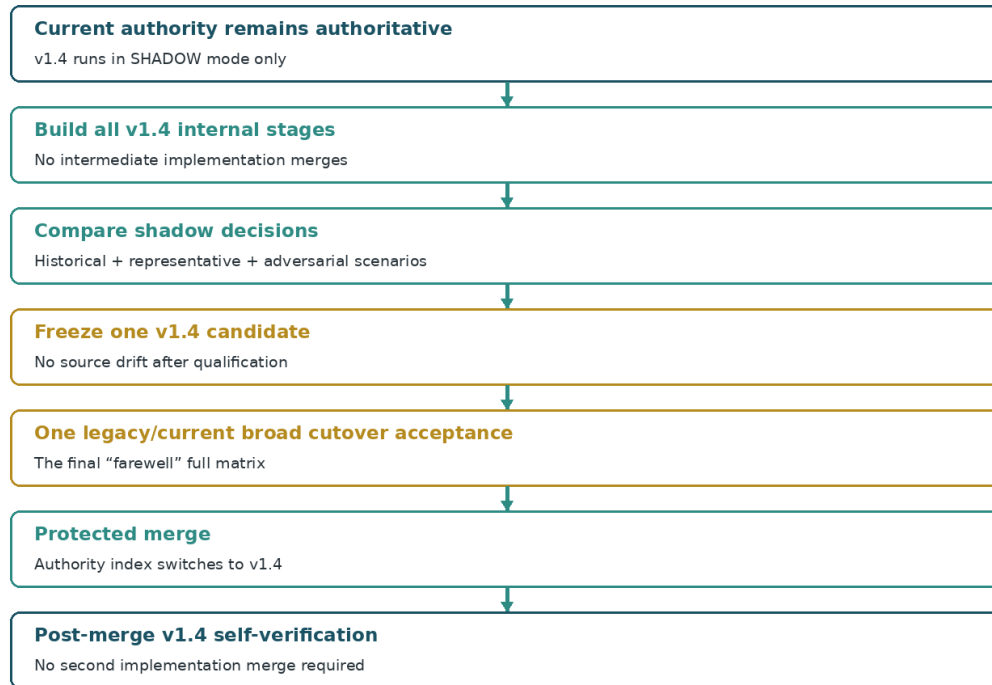


Figure 5. v1.4 is developed under shadow authority and receives one final current-authority cutover acceptance before activation.

31.1 Branch and merge rule

```

origin/main
-> one dedicated Sounding Line v1.4 branch/worktree
  -> Stage 1 baseline / contracts
  -> Stage 2 evidence preservation
  -> Stage 3 impact-selected mainline
  -> Stage 4 prepared layers
  -> Stage 5 lightweight binding
  -> Stage 6 mainline train
  -> Stage 7 shadow/cutover closure
-> ONE implementation PR
-> ONE final current-authority broad cutover acceptance
-> protected merge
-> authority index v1.4 active
  
```

31.2 Why this is a controlled exception

The normal project doctrine remains phase-level integration. This exception exists because these stages jointly replace one authority mechanism and are unsafe as independently authoritative plateaus. Internal commits remain reviewable and revertible; they simply do not become separate protected-main authority states.

32. Rollback and Emergency Serial Mode

v1.4 shall retain a governed emergency path that can execute a conservative broad serial acceptance if the train, preservation engine, prepared-artifact registry, or tree-binding runtime is unavailable or suspected incorrect. This path is not a competing normal authority. It is a fail-closed recovery mode behind the same protected decision semantics.

Failure	Rollback behavior
Train planner defect	Disable train admission; candidates revert to individually planned current-base acceptance.
Preservation defect	Disable preservation for affected obligation class; fresh proof/conservative fallback until repaired.
Prepared artifact defect	Revoke artifact/class; rebuild from trusted source and rerun dependent evidence.
Tree-binding defect	Stop train promotion; use full exact-candidate current-base finalization under emergency policy.
Authority-index/cutover defect	Revert to last accepted authority snapshot through protected Sounding Line infrastructure repair.
Security incident	Revoke shared artifacts/evidence as necessary, preserve incident evidence, and prohibit reuse until requalified.

Rollback cannot fabricate authority

Emergency serial mode may be slower. It may not treat pre-cutover evidence as valid for a changed source merely because the optimized path is unavailable. Every mandatory claim remains source/policy/environment bound.

33. Internal v1.4 Implementation Stages

The implementation program is divided into internal stages for sequencing and evidence, not separate mainline phases.

Stage	Name	Mandatory outcome
0	Freeze the Fleet	Pause active project phases safely; preserve branch/worktree/source/evidence state; do not reconcile/merge them yet.
1	Measure the Channel	Re-measure current mainline mandatory/conditional suite counts and critical path; inventory authority/runtime; freeze v1.4 exact requirements.
2	Preserve the Proof	Implement semantic fingerprints, contract closure, preservation/rebinding, legacy adoption, fail-closed unknown handling.
3	Test What Changed	Implement impact-selected ordinary mainline MSES and sentinel spine; retain exhaustive release-candidate semantics.
4	Prepare the Yard Once	Implement content-addressed prepared artifacts, registry, transport benchmark, scoped worker restoration, DB baseline clone policy.
5	Bind, Don't Retest	Implement tree identity and lightweight protected binding; eliminate redundant heavy binding work.
6	Run the Mainline Train	Implement ordered predicted integration trees, speculative downstream qualification, train brakes/replanning/fairness.
7	Cut Over Authority	Run shadow comparison, adversarial scenarios, one final current-authority broad acceptance, protected merge, post-merge v1.4 self-verification.

33.1 Prompt 0 relationship

Prompt 0 “Measure the Channel” is complete and incorporated into this revision. It surveyed origin/main 0055d012a121a8950b7fa70d371d5eafc6223d10 and produced the Current Authority Survey plus a 30-item top-level Governing Requirements Register. Those records are authoring evidence, not independent authority. Stage 1 SHALL now treat their verified

facts as the baseline, remeasure only material drift, and preserve the amendment's core invariants: proof equivalence, fail-closed preservation, mapping hardening before omission, tree equality, one protected authority, and atomic cutover.

34. Acceptance Matrix and Exit Criteria

Acceptance family	Required proof
Evidence preservation	Unrelated project change preserves eligible evidence; changed contract invalidates; unknown dependency fails closed; derivation chain verifies end-to-end.
Impact-selected mainline	Representative candidates select sentinels + affected obligations only; omitted suites have machine-readable explanations; broad fallback is tagged/debt-bearing.
Prepared layers	Exact identity hit reuses; lock/schema/browser mismatch rebuilds; tamper fails; mutable DB/browser state remains isolated; restore vs rebuild benchmark recorded.
Worker bundling	Multiple logical suites share compatible worker without receipt count/result/cleanup collapse.
Tree identity	Synthetic tree deterministic; physical merge matching tree preserves downstream proof; mismatch brakes train.
Train mutation	Commit/withdrawal/reorder/base advance replan begins at correct earliest car; unaffected upstream evidence survives.
Migration conflict	Collision blocks train rather than silently renaming/reordering.
Protected binding	No heavy re-execution absent semantic drift; exact head/base/tree/policy/evidence/cleanup verified; required check unchanged.
Security	Layer poisoning/replay/substitution/reorder/cache tamper tests fail closed.
Workspace	v1.3 cleanup receipts, disk state, run-owned resources, dormant-worktree slimming remain conformant.
Shadow equivalence	Optimized and current-correct reference paths produce equivalent mandatory decision claims across representative/adversarial corpus.
Atomic cutover	One v1.4 implementation candidate obtains current-authority RELEASE_GO and merges; authority index activates v1.4; post-merge self-verification passes.
Rollback	Emergency serial/current-base fallback demonstrated without competing authority or stale-evidence reuse.

35. Machine-Readable Policy and Registry Changes

Exact filenames may be reconciled by the Stage 1 repository survey, but the repository SHALL have machine-readable equivalents for the following concepts.

Policy/registry	Required content
testing/sounding-line-authority.json	Effective Part I/II/III chain plus v1.4 cross-part amendment, train enabled state, preservation requirements, prepared artifacts, stable protected check.
testing/evidence-fingerprint-policy.json	Canonicalization version, digest algorithms, dependency families, expiry/compatibility rules, unknown behavior.
testing/prepared-artifacts.json	Artifact classes, identity schemas, trusted producer policy, retention/security constraints, transport strategy metadata.
testing/mainline-train-policy.json	Admission, ordering, fairness, merge strategy, train states, replan causes, tree equality, emergency preemption.
testing/release-gates.json	Impact-selected ordinary mainline obligations, sentinels, release-candidate exhaustive policy, record-closure semantics.

Policy/registry	Required content
testing/contracts.json / impact-map equivalent	Protected producer/consumer dependency closure sufficient for semantic invalidation.
testing/suites.json / resources equivalent	Prepared-artifact needs and mutable run-owned resources separated explicitly.

35.1 Suggested authority-index additions

```
{
  "authority": "SOUNDING_LINE",
  "crossPartAmendment": "1.4",
  "semanticCarryForwardRequired": true,
  "impactSelectedMainlineRequired": true,
  "preparedArtifactsRequired": true,
  "treeBoundQualificationRequired": true,
  "mainlineTrain": {
    "enabled": true,
    "headCarPhysicalMergeOnly": true,
    "predictedTreeEqualityRequired": true
  },
  "protectedBinding": {
    "heavyReexecutionByDefault": false
  },
  "requiredCheck": "Sounding Line / Mainline Decision"
}
```

36. Failure and Conformance Codes

Code	Meaning / required response
EVIDENCE_DEPENDENCY_UNKNOWN	Cannot prove closure. Fresh proof or conservative fallback; mapping debt.
EVIDENCE_FINGERPRINT_MISMATCH	Receipt dependencies differ from current obligation. Invalidate.
PRESERVATION_UNSAFE	Derivation cannot establish equivalence. Do not carry forward.
PREPARED_LAYER_IDENTITY_MISMATCH	Restored artifact does not match sealed requirement. Reject and rebuild.
PREPARED_LAYER_TAMPER	Digest/provenance mutation. Security incident; revoke affected artifacts/evidence.
PREPARED_LAYER_SCOPE_VIOLATION	Worker consumes prepared resource not declared by node/bundle.
MAINLINE_SENTINEL_OMITTED	Required fresh sentinel missing. Plan invalid.
MAINLINE_IMPACT_UNEXPLAINED	Broad or omitted evidence has no dependency explanation. Plan/conformance failure.
TRAIN_PREDICTED_TREE_MISMATCH	Physical integration tree differs from qualified tree. Brake and replan downstream.
TRAIN_SOURCE_MUTATED	Car head changed after freeze. Replan that car and downstream.
TRAIN_BASE_ADVANCED	Protected base changed outside expected train transition. Reconcile and replan.
TRAIN_MERGE_CONFLICT	Synthetic merge cannot produce deterministic tree. Resolve source ownership.
TRAIN_MIGRATION_COLLISION	Migration/schema order or identifier conflict. Resolve before qualification.
TRAIN_POLICY_DRIFT	Authority/merge/finalizer policy changed during train. Invalidate affected cars.
TRAIN_REPLAN_REQUIRED	Generic downstream brake with specific reason code attached.

Code	Meaning / required response
PROTECTED_BINDING_HEAVY_REEXECUTION	Binding repeated heavy proof without new semantic invalidation. Infrastructure incident.
LEGACY_EVIDENCE_ADOPTION_INCOMPLETE	Legacy receipt lacks sufficient provenance; rerun unresolved obligations only.

37. Paused-Fleet Evidence Adoption

v1.4 is being introduced while multiple product project phases are deliberately paused. After cutover, those branches resume from their exact preserved checkpoints. They fetch the new mainline, reconcile once, submit existing focused evidence for v1.4 adoption, and rerun only the obligations classified missing or invalidated.

Prompt 0 remote-only inspection covered Helm, Admiralty, Homeport, Bridgework, and Tideglass. It found that older focused evidence is generally adoptable only when immutable receipt, candidate/base, suite, policy/inventory/authority/plan, clean result, adapter/resource identity, and a reconstructable dependency fingerprint are all available. Older receipts frequently lack the final dependency/resource identities. The existing bounded Helm legacy envelope is a precedent for explicit adoption, not authority to grandfather the fleet.

37.1 Resume contract

21. Fetch the v1.4 accepted origin/main and effective authority index.
22. Read v1.4 testing/authority changes; do not restart the project phase from scratch.
23. Reconcile the paused branch/worktree from its recorded checkpoint.
24. Submit legacy v1.3 focused receipts for adoption before launching equivalent fresh suites.
25. Run only INVALIDATED or adoption-incomplete obligations plus current required qualification sentinels.
26. Continue remaining feature implementation using v1.4 local/subsystem behavior.
27. When phase is complete, enter the v1.4 mainline train as a newly frozen candidate.
28. Do not invoke the retired pre-v1.4 ordinary broad mainline workflow except through declared emergency fallback.

Paused work is not disposable

The authority upgrade must preserve both source work and valid proof. A faster testing system that immediately discards every focused test already paid for would be performing an unusually sophisticated reenactment of the problem it was created to solve.

38. Future Option C: Prewarmed Self-Hosted Workers

v1.4 deliberately stops at a host-independent prepared-layer contract. A later amendment may add a prewarmed self-hosted worker pool that keeps trusted toolchains and prepared layers local, reducing runner startup and transfer time further. That future layer must consume the same sealed plans, artifact identities, mutable-resource isolation, per-suite receipts, cleanup, and protected authority.

38.1 v1.4 compatibility requirements

- Prepared artifact manifests are storage/transport neutral.
- Executor interfaces do not assume GitHub-hosted filesystem layout.
- Worker boot identity and trust policy can distinguish hosted and self-hosted classes.
- No prepared layer relies on mutable machine-global state for correctness.
- Lease broker remains authoritative for mutable resources and capacity.
- Self-hosted worker can be removed without changing proof meaning; only execution cost/capacity changes.

The future pool introduces patching, credentials, poisoned-workspace prevention, disk exhaustion, dead-worker recovery, pool scheduling, and host-security obligations. v1.4 shall not hide those costs by declaring a permanent workstation process to be “basically the same thing.”

39. Explicit Non-Goals

- No product feature development for the eight paused projects.
- No redefinition of Drydock Chronicle validation semantics.
- No weakening of Sealed Hold private-content or provider gates.
- No replacement of GitHub branch protection with a custom train control plane.
- No arbitrary auto-merge of conflicts.
- No dynamic skipping of security/privacy/accessibility proof merely to meet latency targets.
- No new generic caching system outside Sounding Line governance.
- No mandate that every suite be bundled; isolation and dependency critical path control bundling.
- No mandate that every candidate train concurrently; small queue or risky shared changes may validly run serially.
- No permanent v1.3 legacy full-matrix normal path after cutover; only governed emergency fallback/history.
- No self-hosted worker fleet requirement in this amendment.

40. References and Final Governing Rule

Normative project sources for this amendment include the accepted Project Sounding Line Part I Software Verification Architecture, Part II Execution Infrastructure and Parallel Runtime, Part III Repository Policy, Codex Governance, and Release Assurance, their accepted amendment chains through the effective repository authority index, and the current Voyagewright Continuous Development and Mainline Integration Standard. This Prompt 0 Integrated Edition additionally incorporates the August 13, 2026 Project Sounding Line v1.4 Current Authority Survey and Project Sounding Line v1.4 Governing Requirements Register as verified authoring evidence. Those two Prompt 0 records do not themselves change runtime authority; this amendment is the governing vehicle that adopts their requirements once accepted.

The immediate architectural basis is especially grounded in Part I v1.2’s Minimum Sufficient Evidence Set, semantic invalidation, prepared-artifact identity, runtime overprovisioning, gate-specific proof, and throughput rules; Part III v1.2’s one protected authority, carry-forward enforcement, mainline merge-lane boundaries, record closure, and performance SLOs; Part III v1.3’s workspace lifecycle, shared immutable cache boundary, cleanup receipts, and post-merge retirement; and Part II’s scheduler, resource broker, durable execution, isolation, cleanup, and distributed-worker architecture.

Final v1.4 Governing Rule

Prove the exact integration state, not the ritual. Preserve evidence only when the complete semantic dependency chain proves it still means the same thing. Prepare immutable execution inputs once and mutable run state separately. Let downstream candidates qualify the future tree, but let only the head car move protected main. When the physical tree matches the qualified tree, bind the exact result and move on. When anything is unknown, changed, mismatched, or untrusted, brake the train and prove it again.

Appendix A. Evidence Fingerprint Schema

The following schema is normative in capability, not exact syntax. Stage 2 may choose JSON Schema, TypeScript schema, Zod, or another repository-standard representation as long as all fields and canonicalization rules are machine-verifiable.

```
EvidenceFingerprint {
  version
  suiteId
  testSetDigest
  protectedContracts[]
  productionClosureDigest
  productionClosureMembers[]
  schemaDigest?
  migrationDigest?
  generatedClientIdentity?
  testDefinitionDigest
  assertionLibraryDigest?
  fixtureDigest?
  baselineIdentity?
  packageLockDigest
  nodeRuntimeIdentity
  toolchainIdentity
  browserIdentity?
  providerIdentity?
  environmentClass?
  soundingLinePolicyDigest
  gatePolicyDigest
  resourceContractDigest
  preparedArtifactIdentities[]
  cleanupContractDigest
}
```

A.1 Fingerprint reconstruction requirements

- Every field used in a preservation decision is retained or reconstructable from immutable evidence.
- Optional fields are optional only because the protected claim does not depend on that class.
- The planner records the rule that excluded an optional dependency class.
- Canonical serialization and digest version are included in the receipt.
- A fingerprint schema upgrade declares backward-compatibility/adoption rules.

Appendix B. Train Receipt and State Schemas

```
MainlineTrain {
  trainId
  policyDigest
  mergeStrategyIdentity
  baseMainCommitSha
  baseMainTreeSha
  createdAt
  orderedCars[]
  fairnessState
  trainStatus
}

TrainCar {
  carId
  projectPhase
  sourceHeadSha
  sourceTreeSha
}
```

```

predictedParentCommitSha
predictedParentTreeSha
predictedIntegrationTreeSha
syntheticMergeEvidenceDigest
planDigest
msesDigest
currentStatus
qualificationReceiptDigests[]
replanGeneration
mutationCounter
admissionTime
readyTime
}

```

B.1 Train brake reasons

Reason	Brake scope
TREE_MISMATCH	Current car + all downstream.
SOURCE_MUTATION	Mutated car + all downstream.
BASE_ADVANCE	First dependency-affected car + downstream.
MERGE_CONFLICT	Conflicting car + downstream until resolution.
MIGRATION_COLLISION	All schema-dependent downstream cars.
POLICY_DRIFT	All cars whose decision semantics are affected.
PREPARED_ARTIFACT_REVOKED	All evidence/cars depending on revoked identity.
SECURITY_INCIDENT	Policy-defined maximum safe scope; default conservative.

Appendix C. Prepared Artifact Identity Examples

C.1 Dependency layer

```

dependencyLayerId = SHA256(
  canonical(package.json)
  + canonical(package-lock.json)
  + nodeVersion
  + npmVersion
  + os
  + architecture
  + nativeDependencyClass
  + installPolicyDigest
)

```

C.2 Prisma layer

```

prismaLayerId = SHA256(
  dependencyLayerId
  + prismaVersion
  + schemaDigest
  + generatorConfigurationDigest
)

```

```
+ targetPlatformIdentity
)
```

C.3 Browser layer

```
browserLayerId = SHA256(
  playwrightVersion
+ browserEngine
+ browserRevision
+ os
+ architecture
+ browserPolicyDigest
)
```

C.4 SQLite baseline

```
sqliteBaselineId = SHA256(
  sqliteSchemaDigest
+ orderedMigrationDigest
+ fixtureBuilderDigest
+ fixtureVersion
+ baselineCertificationPolicyDigest
)
```

The exact formula may be extended by implementation where omitted inputs can change artifact bytes or semantic behavior. It may not be narrowed merely to improve hit rate.

Appendix D. Detailed Acceptance Scenarios

ID / scenario	Setup	Required result
D-01 Unrelated project preservation	Wakebook candidate evidence exists; Admiralty-only source lands; no Wakebook protected dependency changes.	Wakebook unit/component/browser receipts preserved; exact integration/sentinels rerun as required; no blanket full matrix.
D-02 Shared contract invalidation	A shared Wayfarer history projection consumed by Wakebook changes.	Wakebook dependent evidence invalidated through dependency path; unrelated Admiralty evidence preserved.
D-03 Test-only repair	One browser assertion/locator changes.	Owning browser suite and direct decision evidence rerun; unrelated suites preserved.
D-04 Lockfile change	package-lock changes.	Dependency-sensitive receipts/layers invalidated according to declared dependency; new layer built; unaffected environment-independent claims may preserve if policy proves it.
D-05 Prepared layer tamper	Restored dependency layer digest differs.	Layer rejected; security/conformance incident; dependent fresh receipts invalid.
D-06 Layer transfer slower than warm install	Benchmark shows archive restore consistently slower.	Implementation selects alternate certified representation/transport; identity contract unchanged.
D-07 Two train cars	A and B qualified; B validates T2 while A validates T1.	A lands; tree matches T1; B remains valid and proceeds without redoing unaffected proof.
D-08 Head tree mismatch	A physical merge tree differs from predicted T1.	Train brake; B/C proof cannot land; replan from A result/new main.
D-09 Downstream mutation	C receives a commit while A/B green.	C and downstream replan; A/B remain intact.
D-10 Migration collision	B and prior A use colliding migration identifier/order.	B cannot become TRAIN_GREEN until governed resolution; no blind rename.
D-11 External main advance	Security hotfix lands outside train.	Train suspends; semantic reconciliation; preserve unaffected evidence; recompute affected trees.

ID / scenario	Setup	Required result
D-12 Record-only change	Only completion record/index changes after merge.	record-closure gate; no browser/build/database tax absent semantic gate change.
D-13 Legacy evidence adoption	Paused project has v1.3 focused receipts with source/test provenance.	Reconstruct fingerprint, adopt/preserve; rerun only missing dependency obligations.
D-14 Legacy evidence insufficient	Receipt says PASS but fixture/test identity cannot be reconstructed.	Historical only; fresh owning obligation rerun.
D-15 Release candidate	Mainline train was green for all phases.	Release-candidate still executes current release obligations according to release policy.
D-16 Cutover shadow disagreement	v1.4 shadow says GO but current authority exposes a genuinely missing mandatory obligation.	Cutover blocked; repair v1.4; no authority activation.
D-17 Rollback	Post-cutover train bug discovered.	Train disabled; governed current-base serial/fallback acceptance remains safe; same protected authority check.
D-18 Workspace retention	Prepared layers shared; completed phase worktree merged.	Worktree retired/slimmed per v1.3; prepared shared cache remains bounded by artifact policy.

Appendix E. Threat and Abuse Matrix

Threat	Failure mode	Control
Forged carry-forward receipt	Malicious receipt claims unchanged dependencies.	Digest chain, trusted signer/producer, policy digest, prior receipt verification, closure comparison.
Impact-map omission	Sensitive consumer absent from graph.	Unknown/coarse classification, contract-owner review, conformance tests, conservative fallback.
Cache poisoning	Untrusted branch publishes layer under trusted key.	Trusted publisher boundary, immutable digest, namespace/provenance checks.
Artifact substitution	Layer registry points ID to different bytes.	Consume-time digest verification; hard fail/revoke.
Train reorder	Actor moves later candidate ahead.	Order sealed, authority/audit requirement, downstream replan.
Stale downstream evidence	Upstream tree changed after evidence.	Tree match gate + parent/train identity.
Candidate swap	Same project label, different SHA.	Exact sourceHeadSha sealed; mutation invalidates.
Merge-strategy drift	GitHub policy changes merge method.	MergeStrategyIdentity in train policy; replan/invalidate.
Credential leakage	Prepared layer contains token or session.	Build-time/consume-time scans, no-secret manifest, artifact trust policy.
Private fixture leakage	Real Chronicle data becomes shared baseline.	Synthetic fixture requirement and Sealed Hold/private-content scans.
Disk exhaustion by prepared cache	Shared layers accumulate indefinitely.	v1.3 disk-state integration, quotas/eviction, size telemetry, immutable rebuildability.
Receipt case-count collapse in bundle	Worker reports aggregate green despite missing suite.	Per-suite receipt boundary and expected case inventory; hard conformance failure.

Appendix F. Normative v1.4 Closure Checklist

Family	Acceptance requirement
Authority	Cross-part v1.4 accepted through protected mainline and effective authority index updated.

Family	Acceptance requirement
Baseline	Prompt 0/current-state survey records exact pre-cutover gate, worker, timing, policy, and authority truth.
Evidence fingerprint	Canonical schema implemented; dependency closure and derivation chain machine-verifiable.
Invalidation	PRESERVED/REBOUND/INVALIDATED/CONSERVATIVE_FALLBACK decisions expose dependency paths.
Legacy adoption	Paused v1.3 receipts can be adopted safely or narrowly rerun when incomplete.
Mainline MSES	Ordinary mainline selects sentinels + affected/invalidated/missing obligations rather than a standing broad matrix.
Sentinels	Fresh sentinel registry small, explicit, machine-readable, and regression-tested.
Release candidate	Deep release policy remains unchanged in confidence semantics.
Prepared layers	Dependency, Prisma, browser, and SQLite baseline identities implemented or formally dispositioned where not useful.
Transport benchmark	Chosen layer transport is measured against safe alternatives; no archive-format dogma.
Layer security	Trusted producer, digest verification, secret/private scans, retention, revocation, and tamper tests pass.
Worker bundling	Compatible suites can share physical worker while retaining exact independent receipts.
Tree identity	Synthetic integration tree deterministic and final binder compares actual tree.
Protected binding	No routine heavy re-execution after qualified evidence; existing required check retained.
Train admission	Frozen candidates, deterministic parent/tree, migration/conflict checks, and fairness policy enforced.
Train replanning	Mutation, withdrawal, head failure, external main, policy drift, tree mismatch, and migration conflict behave as governed.
Workspace law	Part III v1.3 cleanup receipts, disk-capacity checks, source-only dormant worktrees, and run-owned mutable state remain enforced.
Shadow equivalence	Representative/adversarial comparison proves optimized decisions contain equivalent mandatory claims.
Atomic cutover	One implementation branch/PR obtains one final current-authority broad acceptance and merges once.
Post-merge	v1.4 self-verification passes on actual integrated main without requiring another code merge.
Rollback	Emergency serial/current-base path demonstrated and cannot emit stale or competing authority.
Metrics	Queue/setup/execution/teardown/finalizer, preservation, hit rate, over-invalidation, train and binding metrics emitted.
Performance	Warm ordinary mainline, binding, train-tail, and preservation SLOs measured; misses become incidents not weakened proof.
Option C seam	Self-hosted/prewarmed future workers can consume same plan/layer/resource/receipt interfaces without proof-semantic change.

Amendment acceptance condition

Version 1.4 is fully implemented only when its semantic-preservation, impact-selected gate, prepared-layer, train, tree-binding, shadow/cutover, rollback, and machine-readable enforcement behaviors exist in the repository and pass the acceptance matrix. A PDF that beautifully describes a train while the CI still runs thirty-eight unrelated suites on thirty-eight freshly hydrated workers is not implementation.

Appendix G. Cross-Part Normative Requirements Register

This requirements register is normative. Stable IDs are intended for the machine-readable requirements ledger created during Stage 1. Implementation may split a row into finer requirements but may not silently omit the governed capability.

Requirement ID	Title	Normative requirement	Acceptance proof
SL14-EVID-01	Fingerprint completeness	Every preservation-eligible receipt exposes a canonical evidence dependency fingerprint sufficient to determine whether the protected claim changed.	Schema validation + positive/negative fingerprint fixtures.
SL14-EVID-02	Protected contracts	Every receipt identifies the protected contract IDs/versions and exact logical suite/test inventory.	Receipt conformance tests.
SL14-EVID-03	Production closure	Fingerprint includes registered direct/transitive producer and consumer closure, not only changed file paths.	Impact-map traversal tests.
SL14-EVID-04	Test closure	Test source, assertions, helpers, fixtures, registrations and expected case inventory participate where they affect the claim.	Mutate helper/fixture and prove invalidation.
SL14-EVID-05	Persistence closure	Schema, migration, generated client, baseline and seed semantics participate for persistence-sensitive evidence.	Schema/migration change scenarios.
SL14-EVID-06	Toolchain closure	Lockfile/runtime/browser/compiler/adaptor identity participates only for claims that depend on it, and cannot be omitted when relevant.	Toolchain mismatch matrix.
SL14-EVID-07	Policy closure	Authority, risk floor, gate, resource and finalizer policy identity participates in evidence meaning.	Policy drift test.
SL14-EVID-08	Environment closure	Environment/provider identity is required exactly for environment-sensitive claims and omitted with reason otherwise.	Cross-environment preservation tests.
SL14-EVID-09	Derived evidence immutability	Carry-forward never edits prior receipts; it creates current-source derivation records.	Digest chain verification.
SL14-EVID-10	Exactly one producer	Every mandatory obligation resolves to exactly one current producer after supersession/deduplication.	Finalizer obligation reconciliation.
SL14-INV-01	SHA is not semantic invalidation	A different candidate SHA alone cannot force blanket rerun.	Unrelated-main scenario.
SL14-INV-02	Unknown fails closed	UNKNOWN closure/fingerprint state never qualifies preservation.	Unknown mapping negative test.
SL14-INV-03	Preservation path explanation	Every PRESERVED disposition carries before/after dependency identities and changed-interval digest.	Manifest schema + audit test.
SL14-INV-04	Invalidation path explanation	Every INVALIDATED disposition names the changed dependency path.	Mutation tests across source/schema/fixture/toolchain.
SL14-INV-05	Conservative fallback debt	Broad fallback has reason, unsupported relation, scope and implementation-debt record.	Fallback scenario.
SL14-INV-06	Inverse impact	Planner records why non-selected normally eligible evidence is unaffected.	Plan omission manifest tests.
SL14-INV-07	Base movement	Main advance invalidates only shared/affected/integration obligations, preserving unrelated proof.	Multi-project reconciliation scenario.
SL14-INV-08	Policy movement	Policy/finalizer changes invalidate every receipt whose decision semantics can change.	Authority drift scenario.
SL14-MAIN-01	Impact-selected mainline	Ordinary mainline is computed from obligations and semantic impact, not fixed all-project suite membership.	Plan snapshots for representative projects.
SL14-MAIN-02	Sentinel spine	Fresh sentinel registry is explicit, small, stable, and machine-readable.	Sentinel policy tests.
SL14-MAIN-03	Sentinel anti-abuse	Policy rejects attempts to classify broad unrelated suites as sentinels without explicit governing reason.	Negative policy fixture.

Requirement ID	Title	Normative requirement	Acceptance proof
SL14-MAIN-04	Build policy	Production build freshness is selected according to executable/build impact, while record-only work avoids build tax.	Executable vs record-only plans.
SL14-MAIN-05	Cross-project obligations	Affected producer/consumer boundaries are included even when projects live in different paths.	Contract graph scenario.
SL14-MAIN-06	Missing evidence	A mandatory obligation with no current producer is scheduled fresh or blocks the plan.	Finalizer negative test.
SL14-MAIN-07	Release separation	Release-candidate gate remains independently exhaustive according to current release policy.	Gate comparison test.
SL14-PREP-01	Content-addressed identity	Prepared artifact key is derived from all bytes/semantics that can affect the artifact.	Identity mutation matrix.
SL14-PREP-02	Candidate-independent reuse	Artifacts may be reused across candidates when identity matches; candidate ID itself is not a cache key.	Cross-candidate hit test.
SL14-PREP-03	Immutable publication	Published prepared artifact cannot mutate without digest/provenance failure.	Tamper test.
SL14-PREP-04	Trusted producer	Shared trusted artifacts are produced only by authorized workflow/worker identities.	Untrusted PR publication denial.
SL14-PREP-05	Mutable state separation	Writable DBs, browser contexts, ports, storage roots and process state are never shared merely because prepared layers are shared.	Isolation matrix.
SL14-PREP-06	Dependency layer	Dependency environment identity includes lock/runtime/OS/arch/install/native constraints as required.	Cross-platform identity test.
SL14-PREP-07	Prisma layer	Generated client identity includes dependency layer, Prisma version, schema and generator config.	Schema/generator mutation test.
SL14-PREP-08	Browser layers	Browser identity includes Playwright, exact revision/engine and platform constraints.	Revision mismatch test.
SL14-PREP-09	SQLite baseline	Certified baseline identity includes ordered migrations and deterministic fixture semantics.	Baseline certification/reclone test.
SL14-PREP-10	Transport benchmark	Chosen representation is benchmarked against safe alternatives; slower restore may be replaced without changing identity contract.	Recorded benchmark.
SL14-PREP-11	Retention	Prepared artifacts obey bounded shared-cache retention/size policy and v1.3 capacity state.	Workspace/cache audit.
SL14-PREP-12	Secret safety	Prepared layers contain no real credentials or unauthorized private data.	Secret/private-content scans.
SL14-TREE-01	Candidate head binding	Train car seals exact source head SHA before qualification.	Mutation test.
SL14-TREE-02	Predicted parent binding	Train car seals exact predicted parent commit/tree.	Reorder/base-advance tests.
SL14-TREE-03	Deterministic synthetic tree	Governed merge method yields reproducible predicted integration tree.	Repeat integration fixture.
SL14-TREE-04	Actual tree comparison	Physical landing requires actual integrated tree equality with qualified predicted tree for downstream preservation.	Match/mismatch scenarios.
SL14-TREE-05	Tree not sole authority	Matching tree never bypasses head/base/parent/policy/evidence/cleanup checks.	Candidate substitution negative test.
SL14-TRAIN-01	Head-only physical merge	Only head car may physically advance protected main.	Concurrency/branch-protection test.
SL14-TRAIN-02	Downstream speculation	Non-head cars may execute qualification against predicted trees when resources permit.	Four-car train scenario.
SL14-TRAIN-03	Frozen admission	Candidate cannot be trained while source is still mutable or known required defects exist.	Admission denial test.
SL14-TRAIN-04	Mutation boundary	Changed car and downstream replan; earlier unaffected cars remain valid.	Car C mutation fixture.
SL14-TRAIN-05	Withdrawal boundary	Removed car causes downstream parent/tree recomputation from its position.	Withdrawal fixture.

Requirement ID	Title	Normative requirement	Acceptance proof
SL14-TRAIN-06	Head failure	Failed head cannot block unrelated evidence forever; train reorders/replans under policy.	Head failure scenario.
SL14-TRAIN-07	External main advance	Unexpected protected-main movement brakes promotion and invokes semantic reconciliation.	External advance scenario.
SL14-TRAIN-08	Merge conflicts	Synthetic merge conflict blocks car; no automatic ownership guess.	Conflict fixture.
SL14-TRAIN-09	Migration collisions	Migration/order collision is explicit integration defect and train blocker.	Dual migration fixture.
SL14-TRAIN-10	Policy drift	Merge strategy/authority/finalizer drift invalidates affected train assumptions.	Policy drift fixture.
SL14-TRAIN-11	Fairness aging	Qualified candidates cannot starve indefinitely under a stream of later work.	Scheduler fairness simulation.
SL14-TRAIN-12	Emergency priority	Security/emergency preemption is explicit, audited and causes deterministic replan.	Priority override test.
SL14-BIND-01	Stable required check	Required protected check remains Sounding Line / Mainline Decision.	Branch protection/policy validation.
SL14-BIND-02	Lightweight binding	Binding validates identity/provenance/tree/MSES/conformance rather than routinely rerunning application tests.	Binding timing and command audit.
SL14-BIND-03	Heavy reexecution incident	Unnecessary npm/Prisma/browser/DB/build/test work during binding emits conformance incident.	Injected overexecution scenario.
SL14-BIND-04	Exact MSES	Binding fails if any mandatory obligation lacks one current valid producer.	Missing receipt test.
SL14-BIND-05	Cleanup	Runtime and workspace cleanup evidence remains part of binding decision.	Cleanup failure negative test.
SL14-CUT-01	Shadow authority	v1.4 shadow path cannot emit merge-authoritative decision before cutover.	Shadow RELEASE_GO denial.
SL14-CUT-02	Decision equivalence	Representative optimized plans expose the same mandatory decision claims as current correct authority.	Dual/shadow comparison report.
SL14-CUT-03	Atomic implementation	Stages 1-7 do not become separate mainline authority states.	PR/branch history review.
SL14-CUT-04	Legacy broad cutover	One final current-authority broad acceptance validates the v1.4 implementation candidate.	Accepted run receipt.
SL14-CUT-05	Authority index switch	v1.4 becomes effective only after protected merge and index update.	Post-merge authority check.
SL14-CUT-06	Post-merge self-verification	Actual integrated main passes v1.4 self/conformance verification without another implementation merge.	Integrated SHA receipt.
SL14-CUT-07	Rollback	Emergency serial/current-base acceptance remains available under one protected authority.	Rollback drill.
SL14-WORK-01	v1.3 preservation	Unique staged/unstaged/untracked/unpushed work remains preserve-first.	Workspace policy tests.
SL14-WORK-02	Shared cache location	Prepared immutable caches live outside dormant phase worktrees.	Workspace audit.
SL14-WORK-03	Run-owned mutable state	All DB/storage/browser/process/port state has task/run owner and terminal disposition.	Cleanup receipt.
SL14-WORK-04	Disk thresholds	Prepared cache cannot bypass v1.3 free-space states or heavy-worktree budgets.	Capacity simulation.
SL14-WORK-05	Successful artifact cleanup	Transient success traces/DBs/storage removed after evidence sealing unless retention policy says otherwise.	Closure receipt.

Appendix H. Mainline Train Algorithm and Decision Procedure

The following algorithm is normative in ordering and safety properties. Implementation language and data structures may differ, but the resulting train state must be deterministic for the same repository, policy, candidate set, and merge strategy.

H.1 Train construction algorithm

```

INPUT:
  protected main M
  qualified-ready candidate set Q
  current Sounding Line policy P

1. filter Q for admission validity and frozen source heads
2. order eligible candidates using fairness/priority policy
3. parent = M
4. for each candidate C in order:
  a. verify C source still equals frozen head
  b. synthesize governed integration of parent + C
  c. if conflict/migration/policy failure: mark C blocked; stop or skip only as policy permits
  d. record predicted parent commit/tree and predicted integration tree
  e. compute semantic obligations against predicted tree
  f. search/adopt/preserve/rebind current evidence
  g. schedule unresolved obligations
  h. seal TrainCar record
  i. parent = predicted integration state
5. emit sealed MainlineTrain plan

```

H.2 Landing algorithm

```

WHILE train has cars:
  head = first car
  wait until head == TRAIN_GREEN
  revalidate head source/base/policy/plan
  run protected binding
  perform governed protected merge
  fetch actual main commit/tree
  IF actual tree != head.predictedIntegrationTree:
    brake train
    preserve incident evidence
    replan from first affected car
  ELSE:
    mark head LANDED
    remove head
    for each downstream car:
      confirm its predicted parent chain is still exact
      preserve already-green evidence when valid
    continue

```

H.3 No-skipping principle

A blocked head car may be withdrawn, paused, or reordered only under explicit train policy. The implementation must not silently jump over a candidate while continuing to use predicted downstream trees that included it. Any order change changes parent identity and therefore requires downstream tree recomputation.

H.4 Critical-path optimization

- Schedule downstream evidence as soon as its predicted tree is sealed, not after upstream physical merge.

- Prioritize longest critical-path obligations early when resource fairness permits.
- Use prepared-layer hits to reduce worker bootstrap but do not delay a short suite indefinitely waiting for an optional cache optimization.
- Bundle compatible short suites when it lowers queue/setup critical path and preserves independent receipts.
- Do not overfill the train with candidates whose expected base is likely to churn continuously; admission policy may cap depth according to observed replan rate.

Appendix I. Prepared Layer Operational Lifecycle

Prepared layers have a lifecycle distinct from ordinary run artifacts. They are reproducible but may be expensive enough to retain temporarily as shared infrastructure. Every lifecycle transition remains content-addressed and auditable.

State	Meaning	Required control
MISSING	No trusted artifact exists for requested identity.	Build through trusted producer path or use safe non-layer fallback.
BUILDING	Trusted producer is materializing artifact.	Single-writer/deduplicated build lease where practical; no consumer until sealed.
SEALED	Artifact digest and manifest finalized.	Read-only publication; trusted provenance recorded.
AVAILABLE	Eligible consumers may restore after identity verification.	Consume-time digest and constraints check.
QUARANTINED	Security/integrity/identity concern.	No new consumers; preserve incident evidence.
REVOKED	Known unsafe or incompatible identity.	Invalidate dependent current evidence as policy requires; force rebuild/fresh proof.
EVICTED	Artifact bytes removed for capacity/retention.	Identity/history may remain; rebuild is safe.

I.1 Producer deduplication

When multiple workers request the same missing identity simultaneously, the registry SHOULD deduplicate publication so one trusted producer builds while others wait, execute through an approved fallback, or consume after sealing. This is an efficiency feature, not a global mutable lock: unrelated artifact identities remain independent.

I.2 Consumption procedure

29. Resolve requested artifact identity from the sealed node/bundle.
30. Query registry for trusted AVAILABLE artifact with exact identity and compatible consumer constraints.
31. Verify manifest and artifact digest before materialization.
32. Restore/mount/copy into a run-safe location or read-only shared path according to class.
33. Record restore duration, transfer bytes, hit/miss, producer identity and digest in runtime receipt.
34. Allocate mutable resources separately.
35. If verification fails, quarantine/revoke and rebuild rather than “trying it anyway.”

I.3 Eviction policy

Eviction should consider size, recency, rebuild cost, hit rate, security updates, browser/runtime obsolescence and Part III v1.3 disk-capacity state. Eviction never invalidates historical test receipts whose evidence records the artifact digest; it only means future fresh execution must rebuild or fetch the artifact again. Revocation is different: revocation may invalidate current evidence because the artifact itself is no longer trusted.

Appendix J. GitHub and Branch Protection

Integration Contract

The Mainline Train may coordinate with GitHub, but GitHub remains the protected repository host and branch-protection mechanism. v1.4 does not create a shadow branch authority that can write main without the existing protected decision.

Integration point	Requirement
Required check name	`Sounding Line / Mainline Decision` remains stable unless a separately accepted governance change modifies it.
Workflow invocation	Authoritative acceptance remains explicit frozen-candidate finalization, not an automatic debugger on every development push.
Train metadata	Train ID, car order, candidate SHA, predicted parent/tree and evidence state are persisted outside ephemeral chat context.
PR head drift	A new commit after train freeze invalidates that car and downstream predicted trees; prior check cannot authorize new SHA.
Merge method	Train synthetic integration uses the same governed content-producing strategy as protected physical merge or a deterministic equivalent with exact tree check.
Branch update	Automated branch updates/rebases are not required if a predicted integration tree can be tested safely; any source history mutation remains explicit and audited.
Protected merge	Only head car with valid binding may be merged. Downstream green status is speculative until parent tree is confirmed.
External merge	Unexpected main change triggers train brake/reconcile rather than silently grafting existing train metadata onto new base.
Permissions	Train service/workflow uses least privilege. Evidence/planning jobs do not need broad write permission merely because a later binding job merges.

J.1 PR status presentation

User-facing or operator-facing status may distinguish: DEVELOPMENT, QUALIFYING, TRAIN_READY, TRAIN_POSITION_n, SPECULATIVE_GREEN, HEAD_BINDING, LANDED, BRAKED, REPLAN_REQUIRED, WITHDRAWN. None of these labels may be substituted for the protected final decision. A green downstream car is not “merged” or “mainline accepted” until its actual parent chain is confirmed and it lands.

J.2 Bridgewatch compatibility

Bridgewatch or a future operational surface may display train depth, car positions, evidence freshness, replan reasons, binding state and timing. It remains observational unless separately governed. Train control must not be granted merely because a dashboard can see the queue.

Appendix K. v1.4 Regression and Negative Test Catalog

ID	Regression / negative assertion
K-01	Broad source glob does not invalidate unrelated contract evidence when exact dependency closure proves no relation.
K-02	Removing a registered transitive consumer from impact metadata causes conformance failure or UNKNOWN, not silent preservation.
K-03	Changing a shared fixture invalidates every consuming receipt.
K-04	Changing an E2E locator in one isolated spec does not automatically invalidate all browser families.

ID	Regression / negative assertion
K-05	Changing package-lock invalidates/rebuilds affected dependency layers and environment-sensitive evidence.
K-06	Changing Prisma schema invalidates generated-client and DB-dependent evidence, not pure unrelated static proof.
K-07	Browser layer exact revision mismatch rejects cache hit.
K-08	Tampering with cached layer bytes after publication is detected at consumption.
K-09	Untrusted feature workflow cannot publish a trusted shared dependency layer.
K-10	Dependency layer never contains task-owned DATABASE_URL/session token/private content.
K-11	Bundle worker with missing per-suite receipt fails conformance even if process exits zero.
K-12	Always-fresh sentinel cannot be satisfied by preserved evidence when policy marks it fresh-required.
K-13	Record-only change cannot select broad browser/build/DB proof without semantic escalation.
K-14	Downstream train car cannot physically merge while not head.
K-15	Car commit mutation changes sourceHeadSha and invalidates that car/downstream.
K-16	Candidate withdrawal recomputes downstream predicted parents and trees.
K-17	Predicted tree mismatch after physical merge prevents next car promotion.
K-18	Same tree but wrong candidate head/parent cannot pass protected binding.
K-19	Migration identifier collision blocks train before DB-dependent speculative evidence is accepted.
K-20	Authority/finalizer change mid-train invalidates impacted decision evidence.
K-21	External main hotfix invokes semantic reconciliation and cannot reuse stale predicted parent state.
K-22	Third duplicate broad gate with unchanged production fingerprint triggers overvalidation incident.
K-23	Protected binding that invokes unnecessary Playwright/npm/Prisma after valid evidence triggers incident.
K-24	Legacy v1.3 receipt with missing fixture identity cannot be adopted as preserved.
K-25	Legacy v1.3 receipt with complete reconstructable identity can be adopted without rerunning unchanged product proof.
K-26	Prepared cache eviction causes rebuild, not evidence-integrity failure.
K-27	Prepared cache revocation invalidates dependent current evidence according to policy.
K-28	Disk enters STORAGE_CRITICAL: new prepared artifacts/heavy runs are blocked according to v1.3, unique work preserved.
K-29	Shadow v1.4 disagreement with current authority blocks cutover.
K-30	Post-cutover emergency serial mode uses same protected authority and cannot accept stale derivations.

Appendix L. Operational Handoff and Paused-Fleet Resume Matrix

This matrix governs the immediate post-cutover handoff for the currently paused project swarm. It is intentionally generic about exact phase names because the repository survey and project branches are authoritative for current labels.

Paused branch condition	v1.4 resume action	Evidence action
Clean checkpoint, focused tests passed, no uncommitted work	Fetch v1.4 main, reconcile branch once, continue remaining phase work.	Submit focused receipts for legacy adoption; rerun only invalidated/missing.
Dirty but coherent uncommitted work intentionally preserved	Inspect exact status before main reconciliation; preserve/commit task-owned source as appropriate.	Do not adopt evidence for source that has materially changed since receipt.
Migration created on paused branch	Check v1.4 main migration namespace/order before continuation.	Schema/migration-sensitive receipts re-evaluated; collision blocks continuation until governed resolution.
Known focused failure existed at pause	Resume development verification at smallest failing scope.	No legacy green assumption; requalify after repair.
Branch based several main commits behind	Use semantic reconciliation rather than blanket full rerun.	Preserve unaffected receipts; exact integration proof fresh.
Paused worktree was slimmed to source-only under v1.3	Hydrate only when needed using prepared layers/current dependencies.	Hydration itself does not invalidate source proof; toolchain identity still checked.
Old task processes/resources still exist	Reconcile/clean before broad resume according to workspace policy.	Cleanup state must become valid before phase closure/train admission.

L.1 Recommended resume message contract

```
Fetch current origin/main containing Sounding Line v1.4.
Resolve the current effective Sounding Line authority index.
Resume this existing phase from its recorded checkpoint; do not restart the phase.
Reconcile current main once using semantic invalidation.
Submit prior focused receipts for legacy evidence adoption.
Rerun only obligations classified INVALIDATED, adoption-incomplete, or freshly required by current
qualification policy.
Continue normal development verification.
When the phase is materially complete, freeze one candidate and enter the v1.4 Mainline Train.
Do not use the retired pre-v1.4 broad ordinary mainline path except through declared emergency fallback.
```

Appendix M. v1.4 Implementation Evidence Ledger

Stage 7 SHALL close the amendment with an evidence ledger that lets a later auditor reconstruct the entire authority transition without relying on chat history.

The ledger SHALL include one row or machine-readable record for every Prompt 0 top-level requirement ID SL14-EP-001 through SL14-SH-002, linked to implementation commit(s), conformance test(s), shadow evidence, acceptance receipt, residual debt, and final disposition. It SHALL also include the controlled before/after performance baseline using the Prompt 0 1,180-second / 84.0-worker-minute benchmark and any newer Stage 1 baseline recorded before source work.

Ledger item	Required record
Pre-cutover baseline	Exact origin/main, effective authority versions/digests, gate suite counts, workflow IDs, runner topology, timing baseline.

Ledger item	Required record
Stage commits	Internal v1.4 commit SHAs and scope; no intermediate mainline authority claim.
Fingerprint coverage	Percent/classes of active suites with exact/bounded/coarse/unknown dependency closure; outstanding debt.
Preservation tests	Positive and negative derivation scenarios and digests.
Prepared artifacts	Classes implemented, identity versions, producer policy, benchmark results, hit/miss sample.
Train proof	At least representative multi-car train with mutation/replan/tree-match scenarios.
Binding proof	Command/timing trace proving final binding is identity/evidence work unless semantic drift occurs.
Shadow comparison	Scenario corpus, legacy/current decision, v1.4 shadow decision, differences and dispositions.
Security proof	Cache tamper, replay, candidate substitution, train reorder/tree mismatch, secret/private scan.
Workspace proof	Cleanup receipts, disk state, cache location/size, retired/slimmed worktrees as appropriate.
Cutover acceptance	Exact final v1.4 candidate SHA and current-authority protected RELEASE_GO receipt.
Merge proof	Protected merge commit SHA, actual tree SHA, authority-index digest and required check state.
Post-merge self-proof	v1.4 integrated-main conformance/self-verification receipt.
Rollback drill	Emergency path evidence and scope.
Residual debt	Explicit v1.4 implementation debt, including any conservative-fallback area, with owner and exit criteria.

Closure statement

The v1.4 implementation is complete only when the evidence ledger proves that the repository can consume qualified phase candidates with less redundant work, while every mandatory protected claim remains complete, current, auditable, and bound to the exact integration state. Throughput is the consequence of better proof architecture, not a substitute for proof.

Appendix N. Representative Cross-Project Train Examples

The following examples illustrate how v1.4 should reason about a realistic Voyagewright project swarm. The project names are examples of current contract relationships, not a frozen requirement that these exact phases remain active forever.

N.1 Example: four independent product phases with one shared history seam

M = protected main
A = Drydock candidate
B = Tideglass candidate
C = Wakebook candidate
D = Admiralty candidate

T1 = M + A
T2 = T1 + B
T3 = T2 + C
T4 = T3 + D

Assume Drydock changes only authoring/simulation contracts, Tideglass changes edition-difference contracts, Wakebook consumes the Tideglass history handoff, and Admiralty changes only privileged account operations. The train planner should not treat all four as mutually dependent merely because all touch `src/\*\*`. Instead it identifies the specific boundary between Tideglass and Wakebook, while Drydock and Admiralty remain largely independent for preservation purposes.

Event	Expected v1.4 disposition
Drydock A lands with matching T1 tree	Tideglass/Wakebook/Admiralty evidence survives unless A changed a registered shared contract/toolchain/schema they consume.
Tideglass B changes history-comparison response used by Wakebook	Wakebook C affected contract evidence invalidates/re-executes; Admiralty D can preserve unrelated proof.
Admiralty D changes only admin UI after C is green	No retroactive impact on earlier landed cars; D proves its own affected surface + sentinels.
package-lock changes in B	Prepared dependency layer changes for B/C/D; environment-sensitive evidence invalidates according to dependency identity, not automatically product semantics that do not depend on package runtime.

N.2 Example: shared Prisma schema pressure

Suppose Wakebook C adds a migration consumed by Admiralty D. Even if Admiralty source has not changed, D's predicted parent schema, generated client, certified baseline, and persistence-sensitive tests now depend on C. The train may still qualify D speculatively against T4, but D must use the predicted post-C schema and cannot preserve database evidence from an earlier parent that lacked the migration.

36. Compute C synthetic integration and migration order.
37. Generate/resolve the predicted schema and prepared Prisma/baseline identity for T3.
38. Compute D predicted integration on T3.
39. Invalidate D persistence-sensitive evidence whose prior schema fingerprint differs.
40. Preserve D pure UI/unit evidence whose contract and toolchain closure remain unchanged.
41. After C lands, compare actual T3 tree and migration identity to predicted state before D promotion.

N.3 Example: security hotfix interrupts the train

An emergency security correction may legitimately advance protected main outside the normal train order. v1.4 does not forbid that. It requires the train to stop pretending its old parent still exists. The train brakes, fetches the new main, computes the changed interval, and preserves downstream proof only when the hotfix does not intersect the relevant protected closures. The security fix receives explicit priority authority; chat urgency alone is not a priority class.

Appendix O. Human, Codex, Scheduler, and Release-Authority Responsibilities

v1.4 succeeds only if responsibility remains separated. The fact that Codex can modify files, a scheduler can order jobs, and GitHub can merge a branch does not mean those actors own verification semantics.

Actor	May do	Must not do
Human/project owner	Authorize project scope, owner walkthroughs, emergency priority, governance changes, manual acceptance where required.	Manually declare missing Sounding Line evidence “close enough” or reuse stale receipts without policy.
Codex implementation task	Implement product changes, register tests/contracts, run local/subsystem verification, freeze candidate, submit to train.	Redesign generic Sounding Line infrastructure from a feature branch, launch duplicate full gates, bypass train/binding, or invent cache authority.
Sounding Line planner	Compute obligations, semantic closure, preservation/invalidation, prepared resources, train position and predicted trees.	Change product semantics, omit mandatory obligations for speed, or preserve UNKNOWN dependencies.
Scheduler / broker	Allocate workers/resources, bundle compatible suites, prioritize critical path under policy, throttle for capacity.	Alter evidence meaning, share mutable state without contract, or reorder train outside sealed fairness/priority policy.
Prepared artifact registry	Publish/resolve trusted immutable artifacts and metadata.	Store mutable run truth, private secrets, or authorize release decisions.
Finalizer	Verify MSES, conformance, provenance, cleanup and decision claims.	Merge source or silently manufacture missing evidence.
Protected binder	Bind exact candidate/base/tree/evidence to existing required Mainline Decision and governed merge.	Run broad product tests by default, accept tree mismatch, or merge non-head car.
GitHub branch protection	Enforce stable required check and protected-main write rules.	Interpret product/test semantics beyond the supplied protected decision.
Bridgewatch/observer	Display train/evidence/health/status.	Become a control plane or grant merge/retry authority without separate governance.

O.1 Completion wording

A candidate may be described as TRAIN_GREEN only when it has complete predicted-tree evidence. It may be described as MAINLINE ACCEPTED only after its physical protected merge has landed and integrated-main proof is complete. A downstream green car is not “merged,” and a shadow v1.4 GO is not “authorized.” This distinction must appear in status files, completion reports, Bridgewatch projections, and Codex final messages.

Appendix P. Atomic Cutover Runbook

This runbook is the normative high-level sequence for the single v1.4 implementation merge. Exact commands and workflow IDs belong to current repository implementation records.

P.1 Pre-cutover

42. Confirm all paused product project branches/worktrees have preserved source state and no task is currently attempting authoritative merge.
43. Fetch current protected origin/main and record exact commit/tree, effective Sounding Line authority versions, branch protection and workflow identities.
44. Confirm the completed Prompt 0 Current Authority Survey and Governing Requirements Register are incorporated into the working amendment revision; remeasure any material repository drift since origin/main 0055d012a121a8950b7fa70d371d5eafc6223d10 and record the delta before implementation.
45. Create one dedicated v1.4 branch/worktree from the accepted base; classify it ACTIVE_HEAVY only as needed under v1.3 workspace policy.
46. Implement internal stages with focused self/conformance tests after coherent changes. Do not dispatch current authoritative broad acceptance as a debugging loop.
47. Continuously run v1.4 shadow planning/comparison on the defined corpus without merge authority.

P.2 Candidate freeze

48. Complete all locally/hosted attainable v1.4 implementation, policy, registry, security, train, preservation, layer, binding and rollback tests.
49. Reconcile current protected main exactly once using the current accepted authority and resolve any true source/policy drift.
50. Freeze the candidate SHA. No further source change is allowed without invalidating the cutover attempt.
51. Generate Workspace Cleanup Receipt for temporary development resources while retaining whatever runtime state is needed for the final authoritative acceptance under policy.
52. Create the v1.4 implementation evidence ledger and shadow-equivalence report.

P.3 Final legacy/current-authority acceptance

The frozen v1.4 candidate SHALL be judged by the authority in force before v1.4. This is intentionally the last broad old-style cutover acceptance even if it is expensive. The candidate cannot use its own new train or preservation semantics to waive the acceptance that grants those semantics authority.

```
freeze candidate C
-> explicit current-authority mainline dispatch
-> all current mandatory cutover obligations
-> RELEASE_GO required
-> no source drift
-> protected merge C
-> fetch actual integrated main M1
-> verify remote parity / branch protection / authority index
```

P.4 Post-merge activation

53. Confirm v1.4 authority index/policy is the effective interpretation chain on actual protected main.
54. Run v1.4 integrated-main self/conformance verification, including tree binding, prepared artifact registry access, and emergency fallback readiness.
55. Confirm the old ordinary broad mainline path cannot remain the default parallel authority; retain only governed emergency/historical use as specified.
56. Retire or slim the v1.4 phase worktree after accepted mainline and preservation proof according to Part III v1.3.
57. Resume paused projects with the Appendix L contract and legacy evidence adoption.
58. Observe the first multi-candidate train closely and record any conformance/performance incident without weakening proof.

P.5 Cutover stop conditions

Condition	Action
Current-authority final acceptance fails	Do not merge. Return to focused development, repair smallest failing scope, requalify, then one new cutover attempt.
Shadow comparison has unexplained missing obligation	Block cutover until equivalence defect resolved.
Prepared artifact security/tamper failure	Revoke affected artifacts, repair producer/registry, rerun dependent proof.
Tree/binder cannot prove exact identity	Block cutover; use current authority only.
Disk/workspace enters unsafe v1.3 capacity state	Preserve source, clean governed temporary state, resume only when policy permits.
Unresolved migration/schema conflict with current main	Resolve through owning project/Sounding Line governance before candidate freeze.
Critical impact mapping remains UNKNOWN/INSUFFICIENT for an omission decision	Do not minimize that obligation. Broaden proof or close mapping debt before cutover.
Prompt 0 top-level requirement has no implementation/evidence disposition	Block cutover. Requirement traceability is incomplete.

Appendix Q. Prompt 0 Verified Current-State Baseline

This appendix incorporates the documentation-only Prompt 0 survey performed against origin/main 0055d012a121a8950b7fa70d371d5eafc6223d10 on August 13, 2026. The survey changed no runtime behavior. Its purpose here is to freeze the measured pre-v1.4 state used for architecture, shadow comparison, performance evaluation, and cutover acceptance. If repository truth changes before implementation begins, Stage 1 records a delta; it does not erase this historical baseline.

Q.1 Effective Authority at Survey Time

Concern	Verified value
Effective Part I / II / III	v1.2 / v1.2 / v1.3
Protected check	Sounding Line / Mainline Decision
Only RELEASE_GO emitter	scripts/sounding-line/finalizer.mjs via explicit exact-SHA workflow_dispatch
Mainline path	Plan validates frozen candidate, runs policy/registry/P34 checks, seals plan, dispatches workers, finalizes receipts
Focused repair	Explicit diagnostic/qualification evidence only; suite must exist in sealed plan
Protected binding	PR-triggered artifact/identity verifier; no planning, product execution, or RELEASE_GO
Automatic broad/final closure	None current
Legacy/emergency broad path	None current; local governed commands are not automatically triggered

Q.2 Mainline Gate and Execution Shape

The policy declares 38 required and 14 conditional mainline suites. The current planner selects all 38 required suites and no conditionals, yielding 38 nodes across three fully parallel waves: 18 / 17 / 3. No exclusive nodes appear in ordinary mainline. Conditional omission is not currently supported by change-impact explanation, and no ordinary mainline receipt can be carried forward.

Class	Count / examples	Current behavior
Required wave 0	18	browser.access-sentinel; static.core; major unit families; all always selected
Required wave 1	17	component families, database.sqlite, retirement.matrix-proof, service.admiralty, unit.tideglass
Required wave 2	3	browser.admiralty, browser.helm, build.production
Conditional	14	migration.mysql + 13 focused browser families; omitted without impact rationale
Exclusive	0	No restart-host/external-provider exclusive node in surveyed mainline

Q.3 Measured Throughput Baseline

Representative successful authority run 31728114656 completed in 1,180 seconds wall-clock with 38 workers and 5,042 summed worker-seconds. The measured profile demonstrates that cumulative bootstrap work, not intra-run serialization alone, is the dominant avoidable cost.

Activity	Summed time	Average	Maximum / note
npm ci	1,489 s	39.2 s	79 s
Checkout	1,154 s	30.4 s	40 s

setup-node	715 s	18.8 s	48 s
Suite execution	1,161 s	30.6 s	379 s
Declared preparation	177 s	4.7 s	31 s
Longest worker	488 s total	browser.helm	379 s execution; 32 s npm ci; 24 s prep
Plan job	130 s		npm ci 43 s; registry discovery 13 s
Finalizer job	46 s		finalizer command ~2 s

Q.4 Semantic Evidence Gaps

Current receipts are source-bound and policy/plan-bound but do not yet carry the identities required for safe per-obligation semantic preservation. Every gap below is normative v1.4 work, not optional metadata.

Identity family	Prompt 0 state
Candidate/source SHA, suite, gate, policy, inventory, plan	Present
Qualified base / synthetic merge parents	Present only in binding/envelope
Tree SHA / predicted-vs-actual equality	V1.4 REQUIRED GAP
Test/helper/assertion fingerprint	V1.4 REQUIRED GAP
Contract dependency fingerprint	V1.4 REQUIRED GAP
Fixture/seed + schema/migration identity	V1.4 REQUIRED GAP
package-lock/dependency/Node identity	V1.4 REQUIRED GAP
Browser/toolchain binary identity	V1.4 REQUIRED GAP
Runtime environment, adapter, cleanup provenance	V1.4 REQUIRED GAP

Q.5 Impact-Map Readiness

The survey found 27 unmapped contracts. v1.4 therefore adds a mapping-hardening gate before proof minimization can be trusted globally.

Area	Prompt 0 class	Coverage / evidence	v1.4 rule
Homeport, Admiralty, Bridgewatch, Wakebook, Tideglass	READY	100% current contract-to-suite mapping	Eligible for fine-grained minimization after fingerprint proof.
Wayfarer / Harborlight / Helm	PARTIAL	93% / 90% / 89%; critical gaps remain	Close critical gaps or broaden affected obligations.
Platform foundation / One Voyage / Lanternwake / Sealed Hold	PARTIAL	Approx. 29%-50%; broad owner paths	Use conservative floors until closure is complete.
Drydock / Deepwater / Landfall	INSUFFICIENT	0% contract-map coverage despite critical contracts	No omission based on contract map until hardened.
Breakwater	UNKNOWN	Owner paths/suites exist but no contracts	Absence is not safety; treat as unknown impact.

Q.6 Prepared-Layer Feasibility and Mutable-State Boundary

All five proposed immutable layer classes are feasible under a content-addressed, verified, read-only identity contract. The survey explicitly rejects shared mutable node_modules workspaces, browser contexts/profiles, writable SQLite databases, ports, storage roots, and build outputs. Those remain run-owned and cleanup-governed.

Layer	Prompt 0 measured/current evidence	Required identity / invalidation	v1.4 disposition
Dependency/runtime	npm ci avg 39.2 s per worker	package-lock, Node/npm, OS/arch, install	Highest-priority layer; benchmark restore vs

		flags, package integrity	warm npm ci.
Prisma client	18 current consumers; isolated cost unavailable	schema, generator config, Prisma version, dependency layer, Node/OS/arch	Manifest-hashed generated layer or regenerate-and-compare.
Chromium	4 current consumers; prep max 31 s includes other work	Playwright package/revision, OS/arch, content digest	Exact revision layer; tamper/revision mismatch rebuilds.
WebKit	0 current mainline consumers; unmeasured	same class as Chromium	Support protocol now; measure when used.
Certified SQLite baseline	5 migration/seed consumers; isolated timing unavailable	schema, migrations, seed, Prisma client, SQLite, baseline manifest	Immutable baseline; each mutating test gets run clone.

Q.7 Train, Binding, and Paused-Fleet Conclusions

- Protected binding is already product-test-free; optimization focus is complete tree identity, cleanup provenance, authority-version coherence, and avoidable bootstrap.
- Merge-train feasibility is PARTIAL FOUNDATION, NOT READY. Current synthetic merge identity proves parent composition, not ordered predicted trees or tree equality.
- Historic focused evidence can be adopted only through guarded reconstruction. Older receipts often lack dependency/resource fingerprints; missing identity means RERUN_REQUIRED.
- The existing Helm legacy envelope is a bounded precedent, not a fleet-wide grandfathering mechanism.
- Five material constraints control v1.4: fresh-only ordinary planner; incomplete sealed-plan semantics; asserted rather than proven cleanup; parent composition without tree equality/train order; and conditional-suite omission without impact explanation.

Q.8 Prompt 0 Source Boundary

Prompt 0 inspected AGENTS.md, .agents/testing-workflow.md, the effective Part I/II/III authority PDFs and index, all listed machine-readable policies, current Sounding Line workflows, planner/authority/finalizer/worker-preparation/binding/record-only/envelope scripts, program records, and current remote Actions/PR metadata. Values not observable in those sources remain explicitly unavailable.

Appendix R. Prompt 0 Governing Requirements

Traceability Register

The Prompt 0 Governing Requirements Register contributed 30 stable top-level requirements across sixteen domains. This appendix incorporates them into the governing amendment. The finer-grained Appendix G requirements remain normative detail; Appendix R is the top-level traceability and cutover checklist. An implementation may add finer child requirements but may not retire or silently reinterpret these IDs without a later governing amendment.

ID	Title	Current implementation gap	Required acceptance proof
SL14-EP-001	Obligation producer exclusivity	plan/finalizer cannot accept preserved or rebound producers.	sealed plan maps every obligation to exactly one fresh, preserved, rebound or explicit fail-closed fallback producer.
SL14-EP-002	Immutable derivation chain	no receipt lineage, changed-interval digest or dependency fingerprint.	derived receipt traces current claim to immutable original receipt and reconciliation analysis.
SL14-SI-001	Per-obligation dependency fingerprints	ordinary planner receives no diff and persists no fingerprint.	changed source produces deterministic per-obligation FRESH, PRESERVED, REBOUND or INVALIDATED dispositions.
SL14-SI-002	Fail-closed unknown impact	normal mainline has no path classifier or reason ledger.	unmapped/ambiguous path produces a documented conservative fallback with bounded affected scope and debt record.
SL14-IM-001	Conditional suite selection with explanation	planner always selects required suites and explains every node only as REQUIRED_BY_GATE.	each selected and omitted conditional suite has a sealed, machine-readable impact explanation and contract/path evidence.
SL14-IM-002	Risk floors survive minimization	no impact selector reconciles risk floor with omission.	adversarial changes to protected contracts select the required floor or fail closed.
SL14-RC-001	Exhaustive candidate obligation closure	no explicit v1.4 MSES closure or carry-forward compatibility rule.	candidate plan proves every release obligation is fresh, semantically preserved, rebound, or rejected.
SL14-RC-002	Migration escalation	no migration dependency identity or automatic escalation.	migration/schema/seed change selects the correct migration proof and rejects stale evidence.
SL14-PL-001	Content-addressed prepared layers	all preparation is worker-local and mutable.	layer manifest includes complete identity, digest, provenance, corruption verification and read-only consumption.
SL14-PL-002	Run-owned mutable state	current receipt does not seal clone/profile/cleanup provenance.	concurrent consumers prove unique writable roots and clean disposal while reusing only immutable layers.
SL14-TI-001	Tree identity in authority evidence	no tree SHA is recorded or compared.	plan/envelope/binding contain candidate, base and resulting tree identities and fail closed on mismatch.
SL14-TI-002	GitHub merge tree equality	no post-merge tree proof.	accepted merge records predicted-tree equals actual-tree, independent of commit-object SHA.
SL14-PB-001	Ultra-light evidence-only binding	binding still bootstraps checkout/Node; record-only also runs npm ci.	binding duration budget covers identity, reconciliation, cleanup and final decision only.
SL14-PB-002	Cleanup reconciliation	no run-owned resource disposition receipt is validated.	binding validates sealed cleanup manifests and rejects missing, non-owned or unresolved mutable resources.
SL14-MT-001	Deterministic speculative trees	no train builder or ordered candidate model.	deterministic merge method produces base, car order, candidate head and tree SHA for every car.

Table R-1. Prompt 0 top-level requirements, part 1 of 2.

R.1 Requirements Continued

ID	Title	Current implementation gap	Required acceptance proof
SL14-MT-002	Downstream replan and fail-closed withdrawal	no replacement, conflict or external-main advancement handling.	simulated conflicts, withdrawals and main advances discard affected cars, preserve unaffected evidence only when fingerprints match, and produce a bounded replan record.
SL14-SE-001	Complete provenance and integrity	prepared layers and semantic inputs have no signed/content manifest.	altered receipt, layer, toolchain, fixture or manifest fails verification before decision.
SL14-SE-002	No privilege expansion	future cache/train design has no stated trust boundary.	threat model and negative tests show no anonymous writes, secret retention or untrusted artifact promotion.
SL14-FR-001	Typed failure dispositions	no obligation-level stale/corrupt/impact/worker/train disposition.	each failure class maps to rerun, invalidate, rebind or fail-closed replan without reusing unqualified proof.
SL14-FR-002	Frozen authority boundary	v1.4 needs this rule for train cars and preserved proof.	test scenarios reject repeated authority on an unresolved candidate and require new frozen identity after repair.
SL14-LE-001	Guarded legacy importer	no general adoption schema or fingerprint reconstruction.	importer accepts only complete compatible metadata and labels all missing identity as rerun-required.
SL14-LE-002	Non-retroactive authority	no compatibility matrix by authority version/suite contract.	compatibility table rejects semantic/policy mismatch while retaining historical receipt for audit.
SL14-SM-001	Dual decision comparison before cutover	no shadow planner or decision-difference ledger.	v1.4 and current plans run side by side; every omission is explained, and any stronger current obligation absent from v1.4 blocks rollout.
SL14-SM-002	Shadow performance measurement	queue, artifact, layer transfer and cleanup metrics are absent.	dashboard/report separates execution, preparation, transfer and wait for both authority modes.
SL14-AC-001	Single authority-version switch	no v1.4 versioned compatibility handshake.	one sealed authority digest selects all v1.4 components; mixed versions fail closed.
SL14-AC-002	Cutover preconditions	no v1.4 readiness checklist.	shadow parity, migration tests, security review, rollback rehearsal and protected-main acceptance are all recorded before activation.
SL14-RB-001	Reversible authority cutover	no v1.4 rollback record or evidence compatibility behavior.	rollback restores prior authority atomically, refuses incompatible v1.4 receipts, and preserves historical audit records.
SL14-PF-001	Measured budgets and regressions	no governing critical-path, worker-minute, queue, transfer or binding budgets.	controlled baseline comparison meets declared budgets without lowering mandatory proof or resource isolation.
SL14-SH-001	Portable immutable layer protocol	no transport, attestation or local content-store protocol.	hosted and self-hosted test fixtures consume the same layer manifest and produce equivalent sealed receipts.
SL14-SH-002	Self-hosted trust and cleanup boundary	no self-hosted attestation, scrub or isolation contract.	self-hosted negative tests detect stale mutable state, unowned process/resource and layer tampering, then fail closed.

Table R-2. Prompt 0 top-level requirements, part 2 of 2.

R.2 Requirement Domains

- Evidence Preservation
- Semantic Invalidation
- Impact-Selected Mainline
- Release-Candidate Exhaustiveness
- Prepared Immutable Runtime Layers
- Tree Identity
- Protected Binding

- Mainline Train
- Security / Evidence Integrity
- Failure and Replanning
- Legacy Evidence Adoption
- Shadow Mode
- Atomic Cutover
- Rollback
- Performance Budgets
- Future Self-Hosted Worker Compatibility

R.3 Cutover Traceability Rule

Stage 7 SHALL block authority cutover if any Appendix R requirement lacks one of: ACCEPTED, ACCEPTED_WITH_DECLARED_EXTERNAL_LIMITATION where the governing requirement explicitly permits it, or DEFERRED_BY_LATER_GOVERNING_AMENDMENT. “Implemented somewhere” is not a disposition. Each accepted item must link to exact source identity, tests/evidence, and current-authority cutover receipt.

Cutover discipline

The point of the atomic merge is to make the authority transition simpler, not to make it magical. If the candidate changes, the proof changes. If the proof fails, the candidate does not merge. If the tree differs, the train stops. The last old-style broad gate is allowed to be annoying precisely so future ordinary candidates no longer need to repeat that annoyance without semantic reason.